

基于风险校准与价格信息价值的微网购电调度

摘要

微网购电的关键在于协调分时套利与供电风险：储能能够转移低价电量，而预测不足会触发高价紧急购电。本文建立统一的能量平衡模型，将确定性优化、预测误差校准、日内滚动调整 and 价格信息评价连接起来，形成四层递进的调度方法。

针对第一问，建立十分钟分辨率的购电与储能联合规划，证明在允许无成本弃电时，充放电循环可在保持费用与储电状态不变的条件下消除，将含 144 个二元变量的模型化为等价线性规划。最优日购电量为 59,482.70 kWh，费用 35,126.85 元，比无储能节省 26.90%。

针对第二问，以岭回归预测负载和光伏，采用历史净负载误差的 70%分位数补足风险余量。334 天总费用为 14,132,612.16 元，比同期预测无余量策略降低 26.50%，比岭回归无余量降低 10.01%，说明风险校准是预测转化为经济收益的重要环节。

针对第三问，按午夜计划与最终有效购电量建立非对称结算函数，在新预报到达后重解剩余时域。一月验证选定 50%分位余量及 6、12、18 点更新，全年费用为 13,575,976.78 元，比相同余量下仅用午夜预报节省 1,469,150.15 元，降幅 9.76%。

针对第四问，构建残差 GRU 与非神经网络预测器的同信息对照。GRU 集成在四个发布时刻均取得最低价格 MAE，全期 MAE 为 0.04366 元/kWh，比岭回归降低 8.09%。滚动 GRU 总费为 14,312,347.70 元，比同期价格预测再节省 2,094.78 元。进一步建立共同名义可行域上的真实价格下界，计算得 GRU 计划费用的剩余改进上限为 136,842.64 元，占其名义费用 1.002%。

研究表明，滚动预报和风险余量决定主要节费空间，GRU 进一步改善价格预测；将预测精度、实际费用与最优值差距分开评价，能够准确识别算法的贡献和后续优化重点。

关键词：微网调度；线性规划；风险校准；滚动优化；GRU；信息价值

一 问题重述

题面研究小区光伏、负载、储能与外部电网之间的电力调控 [1]。储能可以在低价或光伏富余时吸收电量，在高价或供电不足时释放电量；调度既要满足负载，也要尽可能节省购电费用。本文依次研究确定性调度、预测误差、日内更新与波动电价四个层次。

第一问给定每天重复的电价、负载和单日光伏预测，要求每天 00:00 制定全天购电计划，且日初日末储电量相同。需报告指定十分钟时段的购电量、全天总量和费用，以及四小时充放电汇总。第二问中负载与光伏逐日变化，正常购电仍按计划量收费，供电缺口通过五倍交易电价的紧急购电补足，需输出 2—12 月每日完整策略及四个指定日期的结果。

第三问进一步利用每日四个发布时刻的光伏预报，决定是否以及何时调整未执行的购电计划，并计入增购和减购费用。第四问使用附件中的实际波动电价，比较不同价格预测器在日前与滚动两类策略下的实际费用。

二 问题分析

第一问的核心是利用价差在 144 个时段之间转移电量。储能效率决定套利门槛，容量与功率决定转移规模。本文先建立完整互斥模型，再证明可消除循环充放电，以线性规划取得全局最优解并解释集中充放电行为。

第二问中，低估净负载需要按五倍电价补购，而高估仍需支付计划费用。因此采用“预测—风险校准—调度”三步法，以历史误差分位数平衡欠购和多购，再通过全年实际费用检验余量收益。

第三问增加了日内重新申报的机会。新预报降低未来供需的不确定性，但增购和减购价格不对称。需要同时决定更新时间、购电调整量和储能动作，并将已执行区间固定。比较不同更新时间的完整费用，才能判断预报更新的净价值。

第四问的价格预测同时影响套利时机与购电规模。本文以残差 GRU 捕捉周期基线之外的变化，与同期预测及岭回归共享历史信息，并将输出接入同一调度器。在此基础上引入真实电价的名义最优下界，既检验预测精度，也量化距离最优计划的剩余空间。

三 模型假设与数据处理

3.1 参数及工作假设

统一参数见表 1。功率上限按微网母线侧定义，所有决策电量使用 kWh。题面“充放电效率为 90%”可能存在单程与往返解释差异，主实验采用两侧各 90%，并对另一解释单独进行敏感性分析。

表 1 储能参数与时间离散

参数	数值	性质
额定容量	12000 kWh	题面给定
允许储电量	1200—10800 kWh	题面给定
最大充放电功率	5000 kW	题面给定
初始储电量	6000 kWh	1 月 1 日 00:00 给定
充电 / 放电效率	0.9 / 0.9	主实验解释
时段长度	1/6 h	十分钟离散

(1) 第一问按模板起点采样，周期曲线以 24:00 值补齐 00:00，功率在十分钟区间内取常值；全年历史序列按记录终点对应十分钟区间均值建模。状态统一定义在区间边界，功率乘 1/6 小时换算为电量。

(2) 不设置题面未给出的售电收入与折旧价格，允许不能使用的电量被弃用。第一、二问普通购电按计划量结算；第三、四问主动调整按第八节的保留量、取消量和新增量分项结算。电池磨损成本没有纳入当前目标。

(3) 实际执行近似每十分钟内功率恒定、当前供需可即时观测并平衡。该近似只服务于区间内的充放电和紧急购电，不允许日前读取当天真实曲线。

(4) 第二问实际 SOC 连续跨日。日前名义轨迹的日末下限取 6000 kWh，作为防止计划耗空电池的策略设置；不强制实际日末回到 6000，也不按天重置电池。

3.2 输入审计与时间边界

附件 1 有 144 个时段，附件 2 的负载与光伏各有 $365 \times 144 = 52,560$ 个观测。读取后检查到完整的 2025 年日期、相同的负载和光伏时间轴，未发现缺失、负数或重复日期，未无依据删除峰值。附件 1 混合存储了时间类型与字符串，统一转换分钟后核对 10、20、…、1440。电量按下式换算：

$$L_{d,t} = P_{d,t}^{\text{load}} \Delta t, \quad V_{d,t} = P_{d,t}^{\text{PV}} \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

重新读取原始工作簿确认：附件 1 的时间列含 60 个时间类型单元格和 84 个字符串，均已规范化；附件 2 的两类序列各 52,560 项无缺失、非有限值或负数，且无重复日期或完全重复的日曲线。全局四分位距规则未标记超界点，但这一检查不能证明不存在局部测量误差。

因此，本次预处理没有执行填补、删点、缩尾或平滑，只执行时间格式与单位规范化。光伏有 23,540 个零值，其本身不是缺失标记，保留在原始记录与回测中。预测值的非负截断属于模型输出规则，训练集标准化属于特征处理，均不等同于修改实际观测。

实际数据的测量不确定性没有单独给出，因此不添加虚构置信区间。全文图线来自原始输入或保存的实际计算结果，时段内不做平滑插值。

四 符号说明

下文沿用表 2 的符号。单日模型省略日期下标，日内区间用 1—144，状态包含 145 个边界点；跨日计算另带日期下标。

表 2 主要符号及单位

符号	含义	单位
d, t	日期与十分钟时段	—
$L_{d,t}, V_{d,t}$	负载与光伏电量	kWh
$\hat{L}_{d,t}, \hat{V}_{d,t}$	日前负载与光伏预测	kWh
$g_{d,t}, e_{d,t}$	计划购电与紧急购电	kWh
$c_{d,t}, d_{d,t}$	母线侧充电与放电	kWh
$w_{d,t}$	不能利用的富余电量	kWh
$S_{d,t}$	区间起点的储电量	kWh
p_t	固定分时交易电价	元/kWh
η_c, η_d	充电与放电效率	—
z_t	充电状态二元变量	—
λ, q	岭惩罚强度与余量分位数	—
$r_{d,t}(q)$	非负净负载预测余量	kWh

日期下标 d 与放电变量的字母 d 通过下标位置区分；为简化实现，程序字段分别使用 `date` 和 `discharge_kwh`。

五 第一问的模型建立与求解

5.1 供需结构与决策目标

已给曲线及固定电价如图 1 所示。光伏集中在白天，而负载覆盖全天，储能调度应同时响应供需差额和分时价格。电价使用独立面板，避免功率与价格共用坐标造成误读。

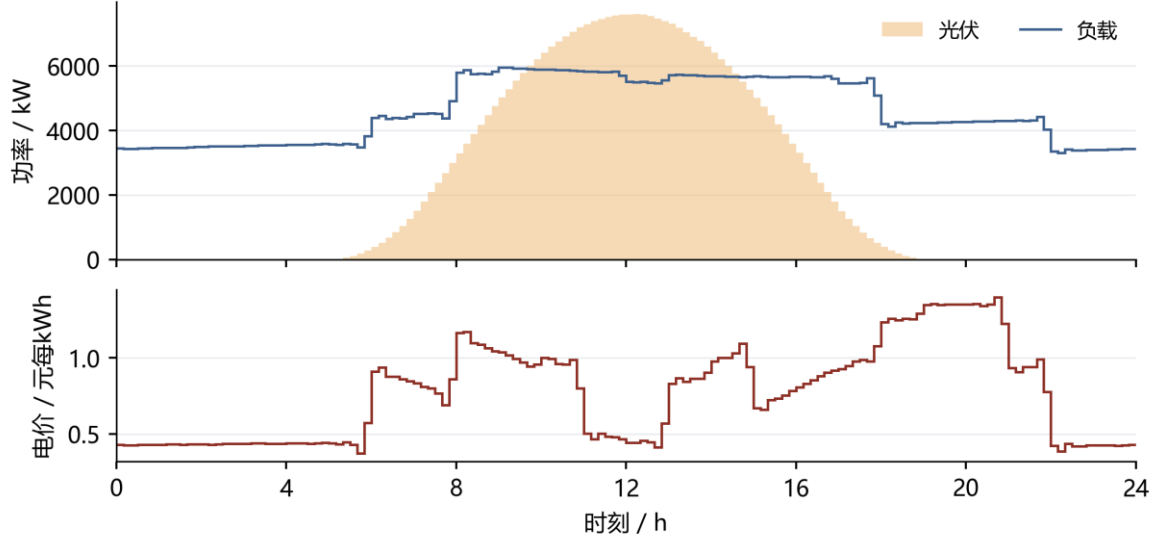


图 1 第一问供需曲线与分时电价

图中每一阶梯对应一个十分钟平均功率或电价，光伏阴影为功率曲线下区域，不作为置信带。

目标是最小化全天计划购电费：

$$\min C_1 = \sum_{t=1}^{144} p_t g_t. \quad (2)$$

以母线侧收支写能量守恒，将允许的富余供电显式记为未利用电量：

$$g_t + V_t + d_t = L_t + c_t + w_t, \quad g_t, w_t \geq 0. \quad (3)$$

充电损耗发生在进入储能时，放电损耗发生在从储能输出至母线时，因而状态方程为：

$$S_{t+1} = S_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad 1200 \leq S_t \leq 10800. \quad (4)$$

SOC 上下界施加于全部 145 个状态点，状态递推与其他时段约束对应 $t=1, \dots, 144$ 。令每时段最大母线侧充放电量 $M=5000/6$ kWh，用二元变量直接限制互斥：

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad z_t \in \{0,1\}. \quad (5)$$

第一问满足以下初始和终端条件，避免利用电池初始存量净放电造成虚假节省：

$$S_1 = S_{145} = 6000. \quad (6)$$

将上述关系集中写成一般日模型，决策变量为购电、充放电、未利用电量、储电量 and 状态变量。给定不同供需输入与初末状态条件时，可复用以下结构：

$$\left\{ \begin{array}{l} \min_{g,c,d,w,S,z} \quad \sum_{t=1}^{144} p_t g_t \\ \text{s. t.} \quad g_t + V_t + d_t = L_t + c_t + w_t, \\ S_{t+1} = S_t + \eta_c c_t - d_t / \eta_d, \\ 1200 \leq S_t \leq 10800, \quad g_t, w_t \geq 0, \\ 0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \\ z_t \in \{0,1\}, \quad S_1 = S_{145} = 6000. \end{array} \right. \quad (7)$$

上述模型有 865 个变量，其中 144 个为二元变量。以 SciPy 1.17.1 的 milp 接口调用 HiGHS [2]，相对间隙目标设为 10^{-7} ，每日求解时间上限为 30 秒，全部运行取得成功求解状态。

5.2 调度结果与指定时段

计算得到全天购电量 59,482.70 kWh、费用 35,126.85 元。购电与储能轨迹见图 2，指定时段购电和四小时充放电分别见表 3、表 4。

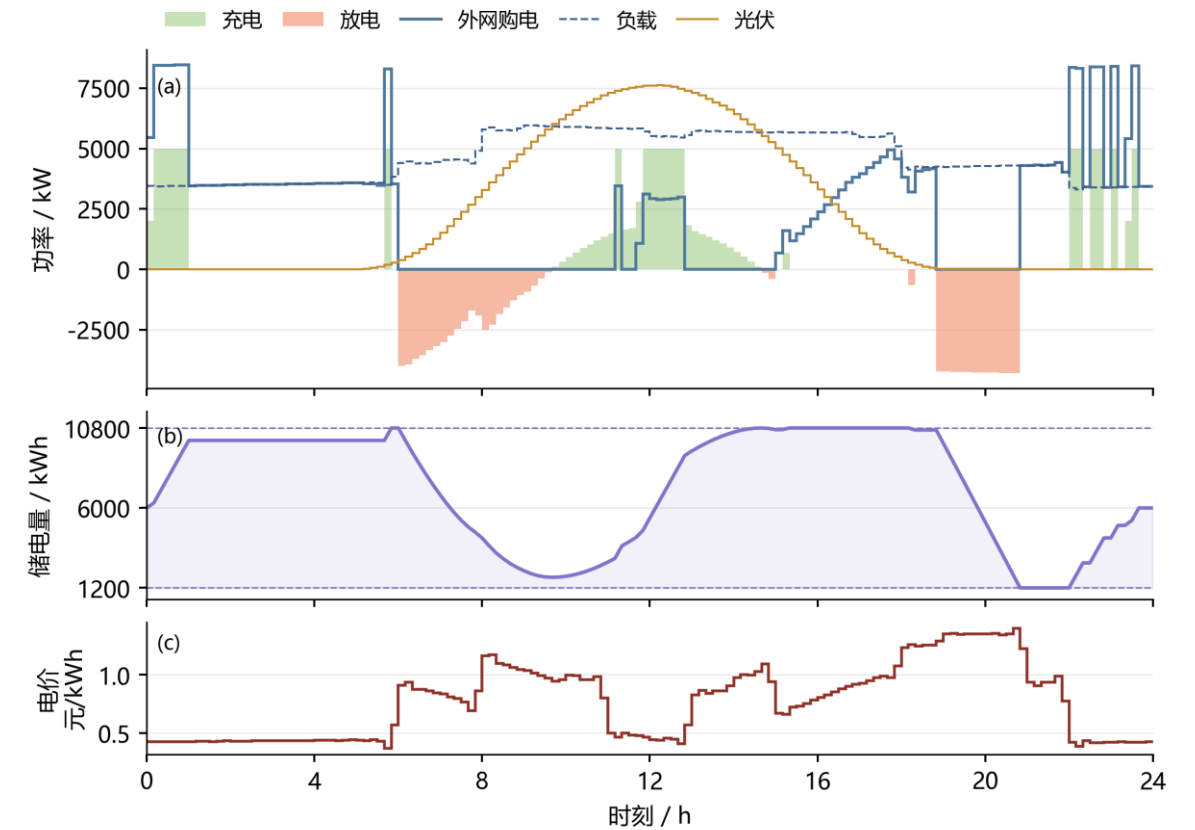


图 2 第一问电价驱动的购电与储能运行

上面板合并供需、购电与充放电功率；中面板显示储电量及上下限，下面板显示电价。共用时间轴保留十分钟决策。

对应结果见表 3。

表 3 第一问指定时段购电量及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量 59,482.70 kWh			全天购电费 35,126.85 元		

对应结果见表 4。

表 4 第一问四小时充放电及边界储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	5,947.42
08:00-12:00	3,954.63	2,121.42	12:00-16:00	6,119.37	91.10
16:00-20:00	0.00	5,068.69	20:00-24:00	5,333.33	3,571.31
00:00 储电量	6,000.00		24:00 储电量	6,000.00	

低价时段的外网购电同时满足负载与充电，形成集中购电峰值；高价时段由储能补充供电，降低外网购电量。图 2 把购电、负载和充放电放在同一功率面板，并以储电状态、电价辅助解释各次动作。表内电量均为 kWh，费用为元；四小时内先充后放不影响十分钟内充放电互斥。

5.3 最优性检验与效率解释

删除整数限制得到线性松弛下界 35,126.84858929 元，与整数解目标之差为 0.00000000 元，求解器相对间隙为零。整数解与松弛下界一致，达到该模型的全局最优费用。能量平衡最大残差约 $2.27\text{e-}13$ kWh，SOC 递推与容量、功率边界均通过复核。

以每时段正净负载直接购电构造无储能基线：

$$g_t^{(0)} = \max(L_t - V_t, 0), \quad c_1^{(0)} = \sum_t p_t g_t^{(0)}. \quad (8)$$

该基线费用为 48,052.05 元，主模型节省 26.90%。效率解释的对照见图 3：若总往返效率为 0.9，并对称取两侧效率为平方根 0.9，费用为 33,801.48 元。提高往返效率可进一步降低套利损耗。

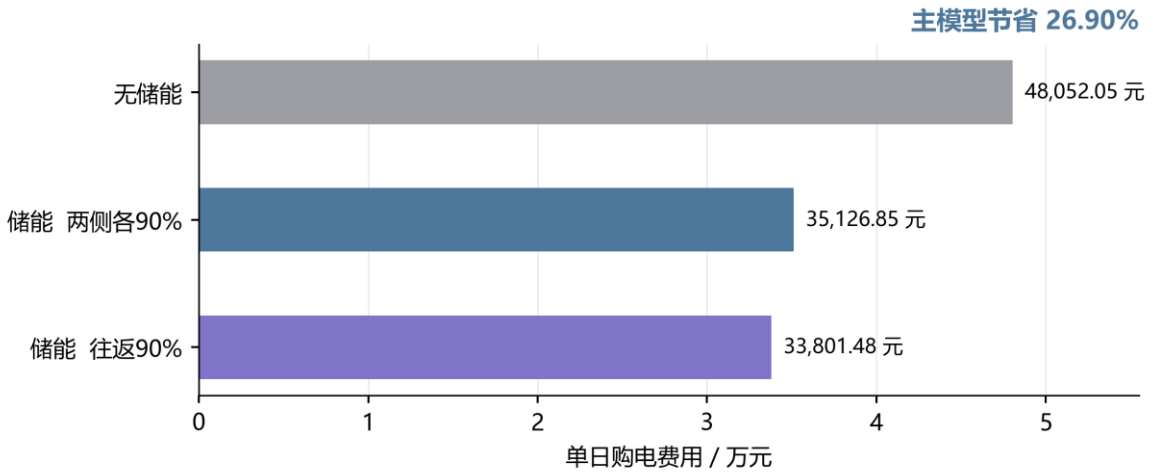


图 3 第一问储能收益与效率解释对照

三组条形从零起算，并在末端标注绝对费用。主文及 Excel 均对应“两侧效率各 90%”。

5.4 互斥约束的等价线性化

互斥不能由一般 LP 自动保证，但本题的自由弃电变量允许给出更强的结构结论。令 $\rho = \eta c \eta d \leq 1$ 。对松弛解任一时段，作如下变换：

$$x_t = \min\{c_t, d_t/\rho\}, \quad c'_t = c_t - x_t, \quad d'_t = d_t - \rho x_t, \quad w'_t = w_t + (1 - \rho)x_t. \quad (9)$$

充电量和放电量均不增加，且至少一个变为零。由于 $\eta c x = \rho x / \eta d$ ，储能增量完全不变；由于 $d' - c' - w' = d - c - w$ ，母线平衡也保持不变。购电、紧急购电和所有计划增减量不变，因此本文各类费用均不变。逐槽处理后得到符合互斥约束的物理解：

$$C_{LP}^* \leq C_{MILP}^* \leq C_{\text{transformed LP}} = C_{LP}^*. \quad (10)$$

因此，在本模型条件下，二者最优费用相等。Q1 的变量可从 865 个减少到 721 个，并去掉 144 个二元变量；日内调整模型同理。实现使用 HiGHS 线性规划，求解后显式消除循环并重新核验物理约束，得到满足充放电互斥的可执行解。

该等价性建立在可自由弃电和线性购电成本上。若加入切换成本或严格弃电限制，应恢复相应约束后重新求解。

六 第二问的预测与日前策略

6.1 训练与评价的时间顺序

以 1 月 22—31 日作为验证窗，选择岭回归惩罚与风险余量。2 月 1 日起固定这些选择规则，执行至 12 月 31 日。模型每七天更新一次，最多使用最近 60 个已结束日期；评价过程中已观察到的数据可以进入后续更新，尚未发生的数据不进入当前决策。这是顺序样本外回测，而不是把全年随机拆分。

一月份共同热身从 1 月 1 日的 6000 kWh 开始；首日无历史时用附件 1 作为冷启动参考，随后不足七天用近三日均值，积累七天后用日 / 周同期。验证期初 SOC 为 6244.256891 kWh，评价期初为 9902.811288 kWh，各候选策略都从同一状态出发。冷启动的附件 1 仅作为明确的初始化假设，不当作全年可提前获得的预报。

6.2 同期基线与岭回归预测

对负载或光伏电量序列 y ，同期基线取昨日与上周同日相同时间槽的均值：

$$\hat{y}_{d,t}^{\text{seasonal}} = \frac{y_{d-1,t} + y_{d-7,t}}{2}. \quad (11)$$

岭回归采用 21 个特征：昨日、前日、上周同日的同槽值，三日和七日同槽均值、七日同槽标准差、昨日全天均值和近三日全天均值共八项；前三阶日内正余弦六项；星期指示七项。日期及时间属于决策时已知的日历信息。标准化均值与尺度仅在当次训练集上估计，常量特征尺度置 1。

$$\min_{\beta, b} \sum_{i \in \mathcal{T}_d} (y_i - b - \tilde{x}_i^\top \beta)^2 + \lambda \|\beta\|_2^2. \quad (12)$$

截距不惩罚，负载和光伏分别拟合。按验证 MAE 比较 $\lambda = 1、10、100$ ，均选 1；实现采用 NumPy 线性方程求解，与带截距岭目标一致 [3]。从 1 月 15 日开始具备拟合条件，之后每七日重新拟合。预测截断为非负；某槽过去七日光伏全零时，该槽预测置零，日出日落季节变化可能使此规则产生滞后。

评价指标采用功率单位的 MAE 与 RMSE；不使用夜间分母为零的普通 MAPE：

$$\text{MAE} = \frac{1}{n} \sum_i |P_i - \hat{P}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (P_i - \hat{P}_i)^2}. \quad (13)$$

6.3 净负载误差余量与名义优化

定义历史净负载误差及只包含过去信息的误差样本池：

$$\varepsilon_{j,s} = (L_{j,s} - V_{j,s}) - (\hat{L}_{j,s} - \hat{V}_{j,s}). \quad (14)$$

$$\mathcal{E}_{d,t} = \{\varepsilon_{j,s} : \max(2, d-28) \leq j < d, 1 \leq s \leq 144, |s-t| \leq 3\}. \quad (15)$$

日期下标以 1 月 1 日为 $d=1$ ；排除首日冷启动误差，最多池化 28 个已结束日期和目标槽前后三槽。跨日边界不绕回，不足 28 天时仅取已有历史。余量规则为：

$$r_{d,t}(q) = \max\{0, Q_q(\mathcal{E}_{d,t})\}. \quad (16)$$

无余量候选直接设 $r=0$ ，与“取零分位数”区分。对 $q=0.7、0.8、0.9$ ，分位数使用样本排序后的线性插值。将预测负载加余量作为名义需求，求解第五节同一物理模型：

$$L_t^{\text{nom}} = \hat{L}_{d,t} + r_{d,t}(q), \quad V_t^{\text{nom}} = \hat{V}_{d,t}, \quad S_{d,145}^{\text{nom}} \geq 6000. \quad (17)$$

70% 分位数不是“全天 70% 概率不缺电”的保证。名义优化只最小化计划费，五倍紧急购电的影响通过验证期对余量方案的筛选间接体现，不能据此称为已精确求解随机规划。

6.4 实时执行与跨日状态

当天普通购电计划固定后，定义当前供需剩余。对非负剩余尽量充电，对缺口尽量放电，并分别受可用空间、储电量和功率限制：

$$u_t = g_t + V_t - L_t. \quad (18)$$

$$c_t = \min\left\{\max(u_t, 0), M, \frac{10800 - S_t}{\eta_c}\right\}. \quad (19)$$

$$d_t = \min\{\max(-u_t, 0), M, \eta_d(S_t - 1200)\}. \quad (20)$$

$$e_t = \max(-u_t - d_t, 0), \quad w_t = \max(u_t - c_t, 0). \quad (21)$$

以上规则使同一时段仅有充电或放电，SOC 仍按状态方程更新，实际日末传给次日：

$$S_{d+1,1} = S_{d,145}. \quad (22)$$

所有供电缺口由紧急购电补足。全年评价费用定义为：

$$C_2 = \sum_{d,t} p_t g_{d,t} + 5 \sum_{d,t} p_t e_{d,t}. \quad (23)$$

将第二问策略概括如下。记 Ω 为第一问中除初末等式之外的物理可行域，输入替换为带余量预测，日初状态取真实已知值；G 表示本节的贪心实时平衡规则：

$$\left\{ \begin{array}{l} (g^{\text{plan}}, \dots) \in \arg \min_{(g,c,d,w,S,z) \in \Omega} \sum_t p_t g_t, \\ L^{\text{nom}} = \hat{L} + r(q), \quad V^{\text{nom}} = \hat{V}, \\ S_1^{\text{nom}} = S_1^{\text{actual}}, \quad S_{145}^{\text{nom}} \geq 6000, \\ (c_t, d_t, e_t, w_t) = \mathcal{G}(g_t^{\text{plan}}, L_t, V_t, S_t), \\ C_2 = \sum_{d,t} p_t (g_{d,t}^{\text{plan}} + 5e_{d,t}). \end{array} \right. \quad (24)$$

其中 Ω 中每一时段的平衡使用上式名义输入；最后一行是实际策略评价费用，不是声称日前已知实际 e 的优化目标。余量 q 仅由一月验证费用选择，之后冻结。

实际储能动作可以偏离名义动作，但普通计划购电不能按真实曲线事后改写。这一贪心控制具有可行性与因果性，不保证跨时段的实时经济最优。

七 第二问的结果与检验

7.1 验证期选择与紧急购电权衡

验证结果见表 5 与图 4。各方案使用相同验证初始 SOC，但期末存量不同，因此除费用外同时列出期末储电量，不把存量差异隐藏在成本比较中。

表 5 一月份十天验证结果

预测器	余量	总费/元	紧急费/元	期末 SOC/kWh
同期	无	661,834.09	173,098.38	9,902.81
同期	70%	634,179.58	110,989.47	10,595.98
同期	80%	666,226.97	64,910.51	10,800.00
同期	90%	680,257.72	456.98	10,800.00
岭回归	无	532,888.41	61,836.07	5,924.08
岭回归	70%	509,757.47	8,765.44	6,623.17
岭回归	80%	529,846.45	4.16	7,110.16
岭回归	90%	633,664.05	0.00	9,039.38

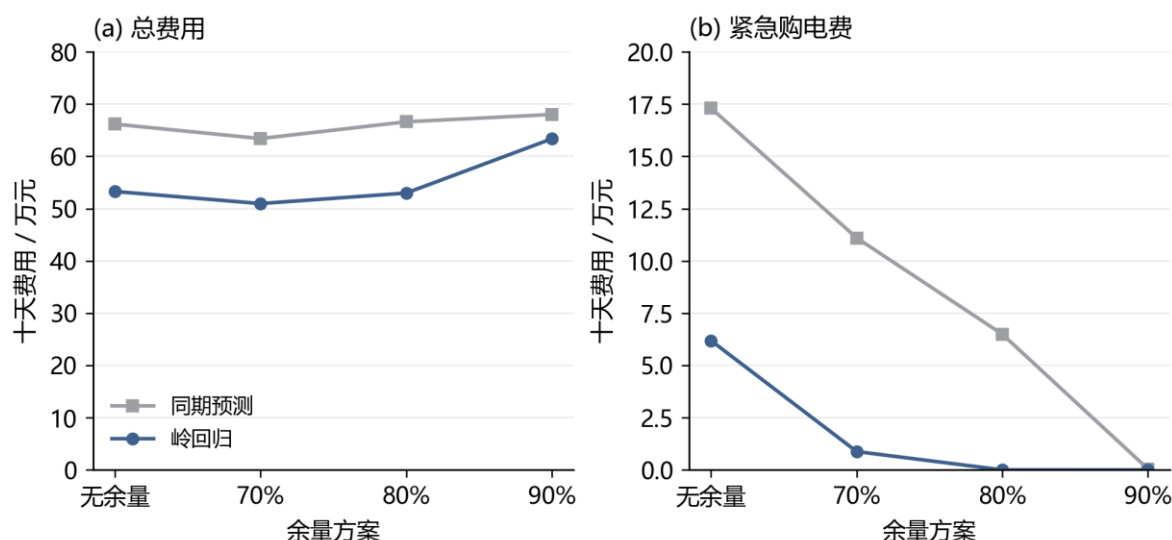


图 4 验证期风险余量与费用的权衡

横轴为四种候选方案，其中“无余量”是独立基线。折线只辅助比较候选值，不表示连续参数的精确响应。

岭回归 70% 余量的验证费为 509,757.47 元，其中紧急费 8,765.44 元；90% 余量紧急费为零，但总费升至 633,664.05 元。两者期末储电量相差约 2416 kWh，故不能完全忽略存量影响；当前目标仍按题面购电费用评价，不另添加缺乏依据的残值收入。

7.2 全期预测误差与季节变化

按表 6 比较 334 天共 48,096 个十分钟区间的功率预测误差，岭回归在负载预测上的改善大于光伏。四个指定日期的曲线对照见图 5，整年误差仍以全期统计为准。

表 6 第二问全期预测误差

预测器	负载 MAE	负载 RMSE	光伏 MAE	光伏 RMSE
同期	372.26	585.91	171.98	343.38
岭回归	179.86	234.59	155.28	306.74

上述误差单位均为 kW，光伏指标包含夜间零值，不能直接等同于白天发电时段的预测精度。

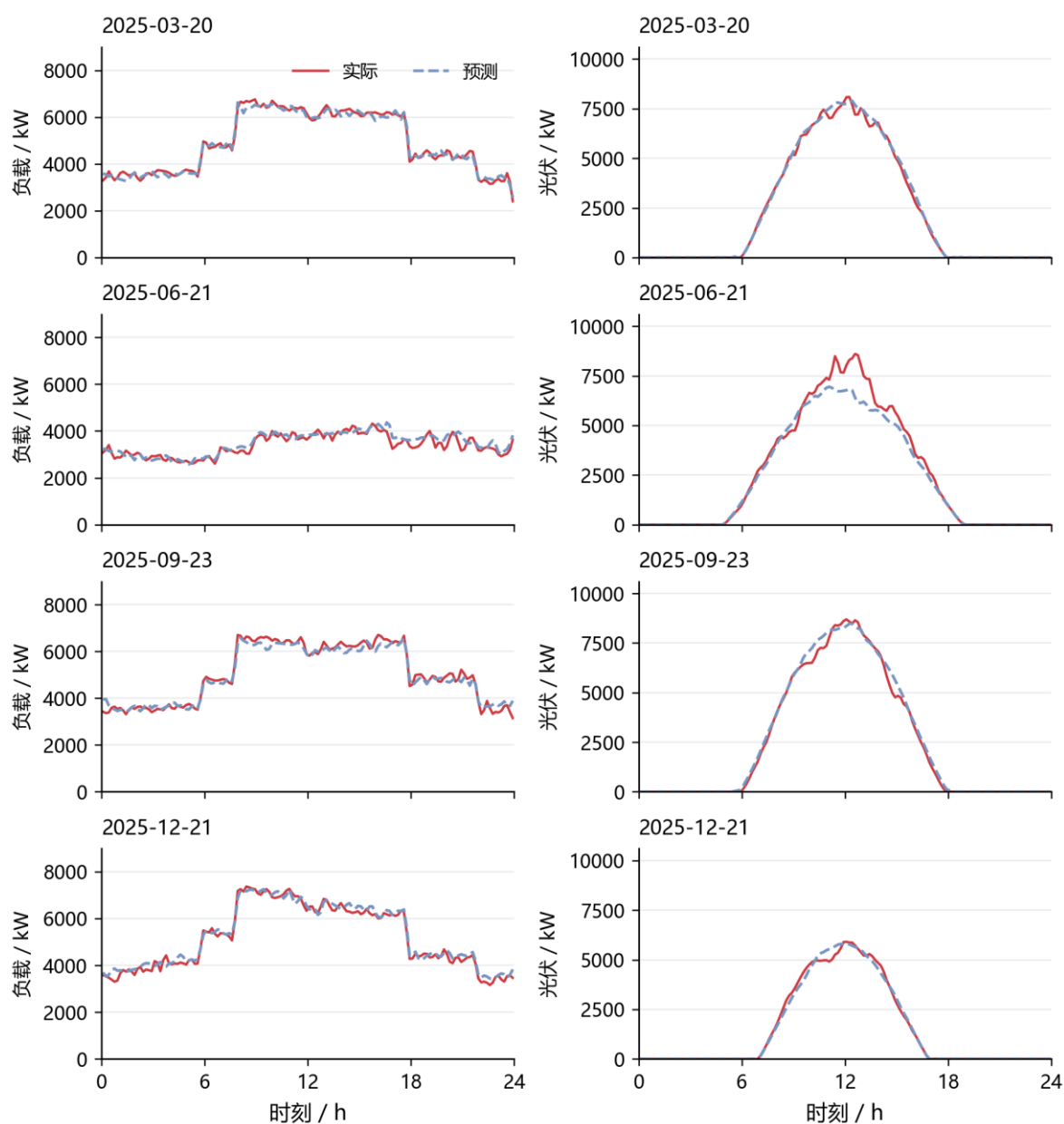


图 5 四个指定日期的负载与光伏日前预测

各列共用纵轴尺度，日期由题目指定。实线为实际观测，虚线为岭回归预测，展示十分钟分辨率下的预测偏差。

7.3 全期购电费用与运行约束

策略费用分解如图 6，数值及期末存量见表 7。相同初始状态保证比较具有共同起点，实际状态在全年内自行连续演化。

表 7 第二问全期费用及期末储电量

策略	计划费/万元	紧急费/万元	总费/万元	期末 SOC/kWh
同期 无余量	1,199.19	723.59	1,922.78	5,825.99
岭回归 无余量	1,215.92	354.62	1,570.54	5,903.65
同期 70%余量	1,297.86	452.54	1,750.40	6,217.35
岭回归 70%余量	1,293.50	119.76	1,413.26	5,903.65

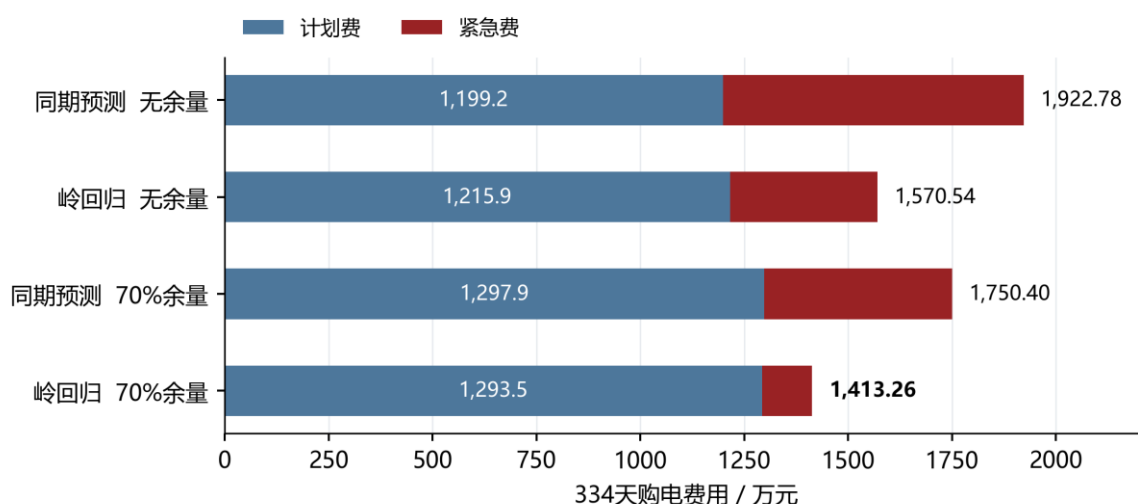


图 6 第二问四组策略的全期费用分解

蓝色为普通计划购电费，深红为紧急购电费；条形右端标注总费用。主策略由一月份验证选择。

主策略总费用 14,132,612.16 元，较同期无余量低 26.50%，较岭回归无余量低 10.01%。后两组期末 SOC 均为 5,903.65 kWh，因此此处的余量增益没有被不同期末存量混淆。

主策略紧急购电累计 197,465.10 kWh，涉及 1132 个十分钟区间，不等同于同样数量的独立事件。未利用电量为 2,211,229.49 kWh，其中可包含未消纳光伏和富余计划购电，不能全部称为弃光。

四个指定日期的购电、充放电和紧急购电结果见附录 A。

八 第三问 日内预报的滚动价值

8.1 预报信息与决策时域

附件 3 含 365 天、每日 4 个版本、每版 24 个整点光伏预报，共 35,040 个值。按四行组还原 1,095 个结构性空日期后，未发现缺失、非有限数或负值。预报发布时间分别为 0、6、12、18 点；时刻 τ 的新预报只服务于 τ 之后的调度。小时预报端点之间采用分段线性函数，并在十分钟区间上积分为预测电量；发布点以最近完成区间的光伏观测衔接。

这一处理保留了预报更新带来的信息差异。每次求解均以当时真实储电量为起点，固定已经执行的购电与充放电量，重新配置其余区间。该时域推进方式与储能模型预测控制的基本框架一致[4]。

图 7 展示指定日期的实际光伏及各版预报；曲线从各自发布时间开始。较晚预报能够改变午后供需判断，为调整计划提供新依据。

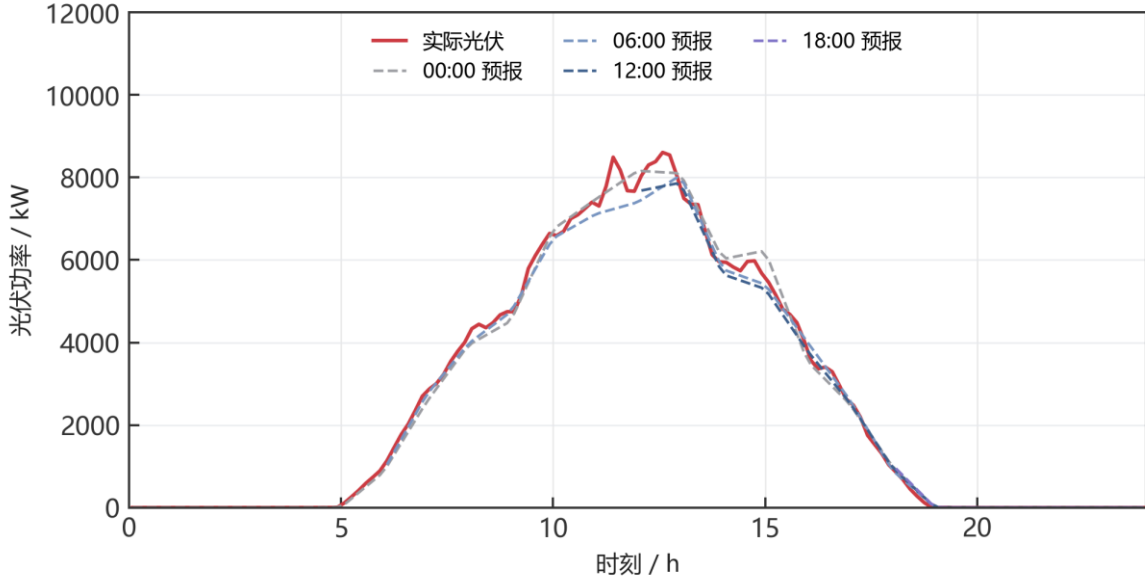


图 7 四个发布版本的光伏预报

8.2 保留量 取消量与新增量的结算

记午夜计划为 x_t ，时段执行前最后一次有效调整量为 y_t ，紧急购电为 e_t 。取消量 u_t 与新增量 v_t 分别定义为：

$$u_t = (x_t - y_t)^+, \quad v_t = (y_t - x_t)^+, \quad y_t = x_t + v_t - u_t. \quad (25)$$

保留购电按原价结算，取消部分按半价支付违约金，新增部分按 1.5 倍价格购买。因此每个区间的完整费用为：

$$F_t = p_t \min(x_t, y_t) + 0.5p_t u_t + 1.5p_t v_t + 5p_t e_t = p_t x_t - 0.5p_t u_t + 1.5p_t v_t + 5p_t e_t. \quad (26)$$

原计划 100 kWh、调整为 80 kWh、电价 1 元/kWh 时，总费用为 $80 + 0.5 \times 20 = 90$ 元；调整为 120 kWh 时，费用为 $100 + 1.5 \times 20 = 130$ 元。前式按保留量计普通费用，后式按原计划记账，两种表达完全等价。多次预报更新均相对同一午夜计划确定偏差；中途计划用于运行与追溯，不重复累计尚未交付区间的修改费用。

8.3 滚动优化模型与风险校准

记 T_τ 为当前尚未执行的区间集合， Ω 为第一问去除首末状态等式后的物理可行域。以更新后的光伏预测、负载预测和对应误差余量作为供需输入，求解：

$$\begin{cases} \min_{y,c,d,w,S,u,v} & \sum_{t \in T_\tau} \hat{p}_{\tau,t} (1.5v_t - 0.5u_t) \\ \text{s.t.} & y_t = x_t + v_t - u_t, \quad 0 \leq u_t \leq x_t, \quad v_t \geq 0, \\ & y_t + \hat{v}_{\tau,t} + d_t = \hat{L}_{\tau,t} + r_{\tau,t} + c_t + w_t, \\ & (y, c, d, w, S) \in \Omega, \quad S_\tau = S_\tau^{\text{actual}}, \quad S_{145} \geq 6000. \end{cases} \quad (27)$$

目标函数省略了与当前决策无关的午夜合同常数项。第一至三问的 p 由附件 1 给定；第四问替换为当时可获得的价格预测。名义规划满足供电需求，实际执行按第二问的实时平衡规则应对残余预测误差。

风险余量使用此前 28 个完整日期、相同发布版本和目标前后三个区间的净负载误差。比较 0、50%、70% 三个分位数与 8 种更新时间组合，共 24 组。一月 22—31 日选择 50% 分位数及 6、12、18 点更新，随后运行 2—12 月。负载预测器和日末名义目标保持一致，使比较集中反映预报更新与调整制度的作用。

图 8 展示一月验证费用和全年累计收益。左图用于确定参数，右图分别比较固定电价下午夜与滚动策略、波动电价下日前与滚动主策略。

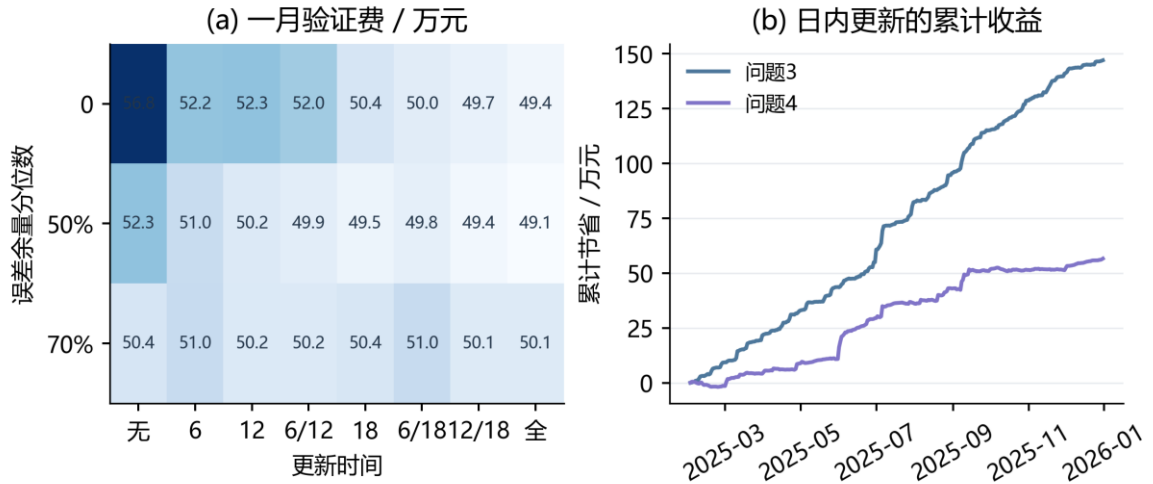


图 8 风险余量选择与滚动收益积累

8.4 更新时间的收益与费用构成

图 9 展示同一 50% 余量下的全部更新时间组合。深蓝标出验证期选定的主方案，红色为紧急购电费用。

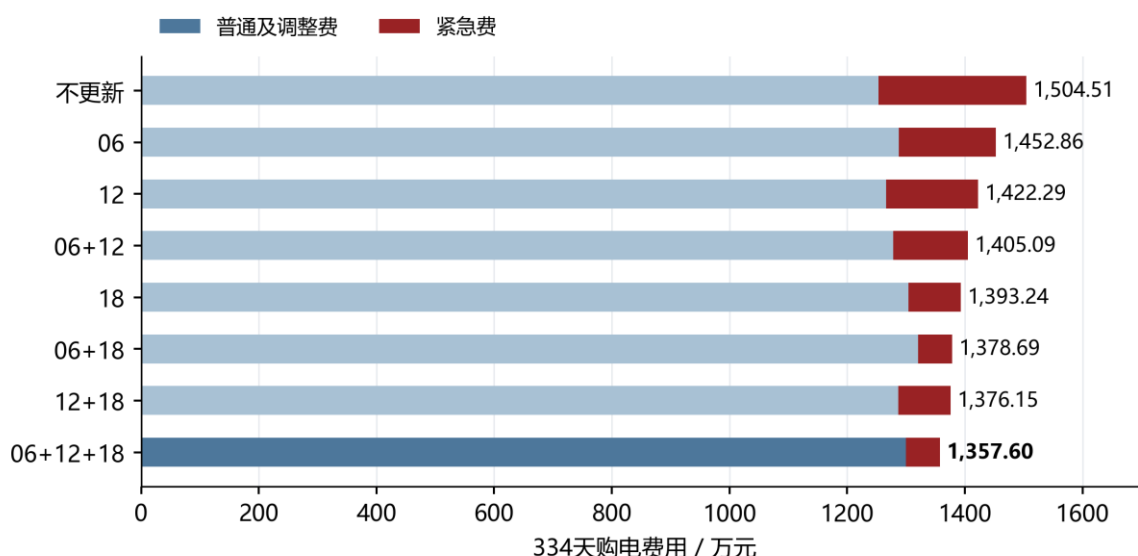


图 9 八种更新时间组合的费用对比

对应结果见表 8。

表 8 第三问主策略与仅午夜预报的费用分解

费用或运行量	仅午夜预报	日内滚动
原计划费/元	12,530,137.77	12,521,239.63
新增购电费/元	0.00	678,211.30
取消量净费用/元	0.00	-203,600.86
紧急购电费/元	2,514,989.16	580,126.71
总费用/元	15,045,126.93	13,575,976.78
紧急电量/kWh	407,324.31	101,019.32
期末储电量/kWh	5,903.65	5,903.65

全年总费用由 15,045,126.93 元降至 13,575,976.78 元，减少 1,469,150.15 元，降幅 9.76%。实际紧急电量降至 101,019.32 kWh。三次更新把部分五倍电价的紧急补购提前转化为常规调整，节省额超过调整成本，因而有必要引入零点之外的预报。两方案初末储电状态相同，净节费不依赖消耗更多期末库存。

九 第四问 GRU 预测与价格信息的优化空间

9.1 波动电价与共同预测信息

附件 4 给出 365 天、每天 144 个电价，共 52,560 点，范围为 0.0076—1.7936 元/kWh。所有价格保留原值，不对峰谷截尾。各预测器使用发布前的价格历史以及已知日历信息，输出未来 144 个十分钟目标；当日调度只读取其中尚未执行的区间。

图 10 展示日均价格与每日最小—最大范围。稳定的日内结构与随机幅度共同影响购电的时机和规模。

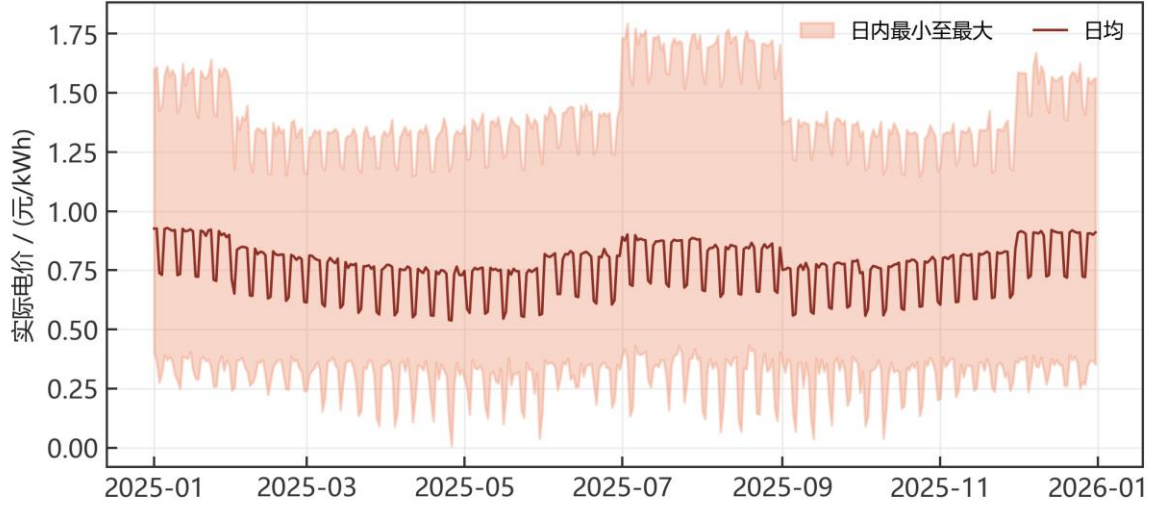


图 10 全年实际电价的日内范围

同期预测取昨日与上周同目标区间价格的平均值。岭回归在这两组历史价格、日历周期特征及滞后统计量上构造 12 维输入。残差 GRU 使用 6 维输入：昨日价、上周价、日内正余弦和星期正余弦。三类预测器共享历史可见范围，区别在于对非线性依赖的表示能力。

$$\hat{p}_{\tau,j}^{\text{seasonal}} = \frac{p_{\tau+j-144} + p_{\tau+j-1008}}{2}, \quad j = 0, \dots, 143. \quad (28)$$

9.2 残差 GRU 的结构与训练

GRU 以门控结构选择保留的历史信息[6]。本文采用单层 32 维隐藏状态，在同期基线之上预测残差，输出头从零初始化，使初始预测与同期方法一致。输入维度较小、序列固定为 144 步，有利于在有限历史数据上控制训练规模。网络按 PyTorch 的 GRU 形式实现[5]：

$$r_j = \sigma(W_{ir}x_j + b_{ir} + W_{hr}h_{j-1} + b_{hr}), \quad z_j = \sigma(W_{iz}x_j + b_{iz} + W_{hz}h_{j-1} + b_{hz}). \quad (29)$$

$$n_j = \tanh(W_{in}x_j + b_{in} + r_j \odot (W_{hn}h_{j-1} + b_{hn})), \quad h_j = (1 - z_j) \odot n_j + z_j \odot h_{j-1}. \quad (30)$$

$$\hat{p}_{\tau,j} = \max\{0.001, \hat{p}_{\tau,j}^{\text{seasonal}} + s_y(w^T h_j + b)\}. \quad (31)$$

前两个价格特征的均值和标准差仅从训练集估计， s_y 为训练价格标准差。网络采用 SmoothL1 损失与 Adam 优化器，学习率 0.003，批量 32，梯度范数截断为 1；最多训练 50 轮，内部验证连续 6 轮无改进即早停。输入窗口每 6 小时取样，只有全部预测目标已发生的窗口进入训练。

1 月 15 日、22 日和之后每月首日重新训练，每次最多使用此前 60 天数据；内部验证为最近 3 个完整日期，跨越训练—验证边界的目标窗口不进入训练。三个随机种子 17、42、2026 分别训练，最终等权平均，共 39 次拟合，CPU 实测训练约 126.56 秒。图中的训练曲线展示一次月初拟合的内部验证过程。

图 11 展示同一月初拟合中三个网络随训练轮数变化的内部验证 MAE；早停分别保留各自最优权重。

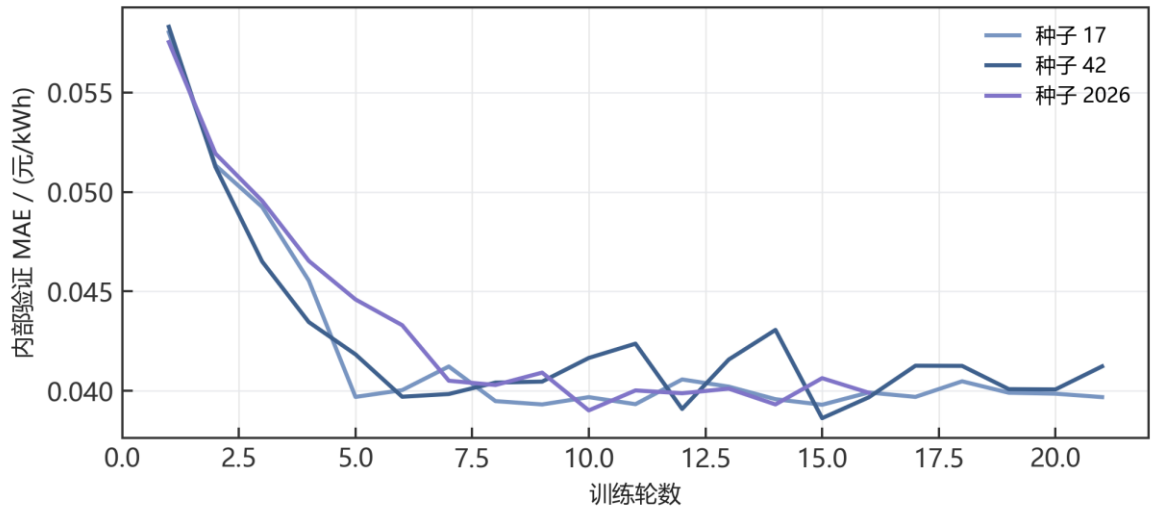


图 11 三个随机种子的 GRU 验证误差

9.3 四个发布时刻的预测优势

图 12 展示各发布时刻相同目标范围上的 MAE。GRU 在四组比较中均取得最低误差。

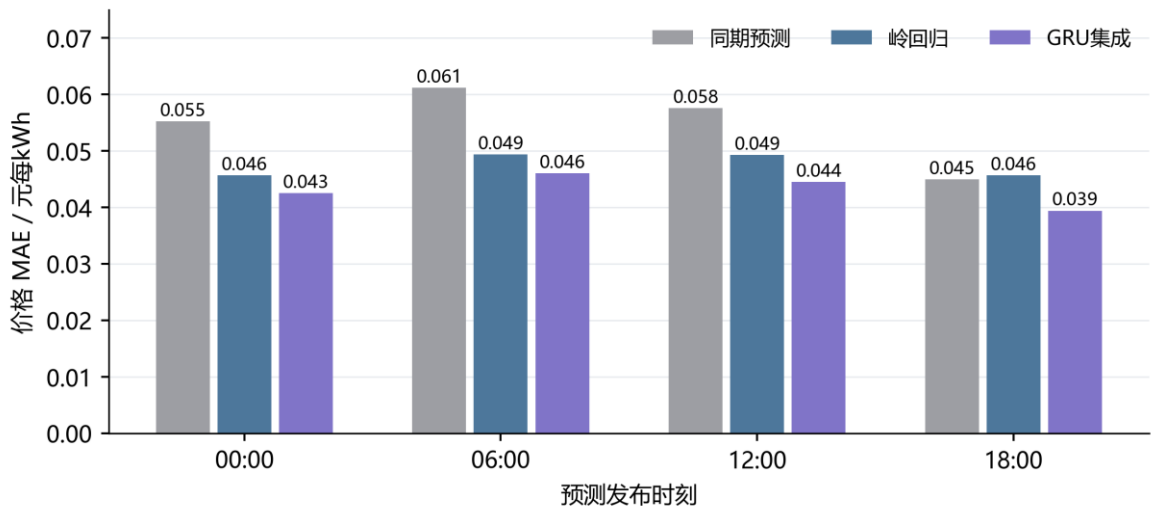


图 12 GRU 与非神经网络预测的误差对比

对应结果见表 9。

表 9 各发布时刻价格 MAE 及 GRU 相对岭回归的改进

发布时刻	同期	岭回归	GRU 集成	改进比例
00:00	0.05527	0.04567	0.04253	6.88%
06:00	0.06117	0.04938	0.04603	6.78%
12:00	0.05761	0.04928	0.04448	9.74%
18:00	0.04499	0.04566	0.03941	13.68%

按全部发布一目标配对加权，GRU 集成 MAE 为 0.04366 元/kWh，较同期预测降低 22.70%，较岭回归降低 8.09%。共 120,240 个配对，其中同一目标可由不同发布时刻预测；分时评价保证各模型比较的是相同信息时点。

图 13 展示实际价与岭回归、GRU 预测的逐时对照。GRU 主要修正同期结构之外的变化，部分突变仍由实时运行承担。

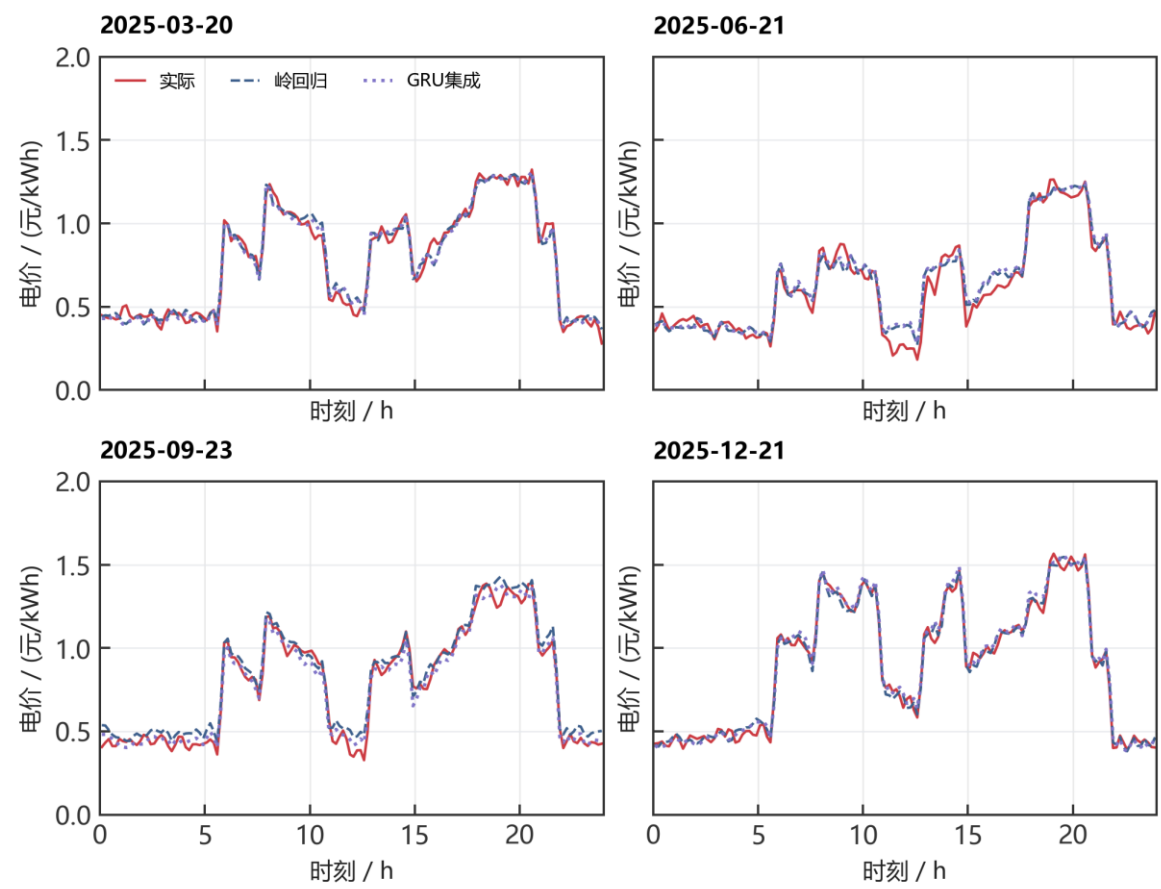


图 13 四个指定日期的午夜价格预测

9.4 预测优势如何转化为费用收益

对应结果见表 10。

表 10 波动电价下两类策略的全年费用

价格预测器	日前总费/元	滚动总费/元
同期预测	14,867,653.20	14,314,442.48
岭回归	14,880,535.05	14,320,233.59
GRU 集成	14,877,217.55	14,312,347.70
真实电价对照	14,818,406.37	14,212,407.27

一月实际费用选择的日前主策略为岭回归，滚动主策略为同期预测，全年分别为 14,880,535.05 元和 14,314,442.48 元，滚动方案降低 566,092.57 元（3.80%）。保持同一滚动策略，GRU 将总费进一步降至 14,312,347.70 元，比同期预测节省 2,094.78 元，比岭回归节省 7,885.90 元。前者是策略层面的改进，后者是价格预测模块的增益。

图 14 展示相对同期价格预测的节省额。零点右侧表示节省、左侧表示增加费用；真实电价对照只用于事后评价。

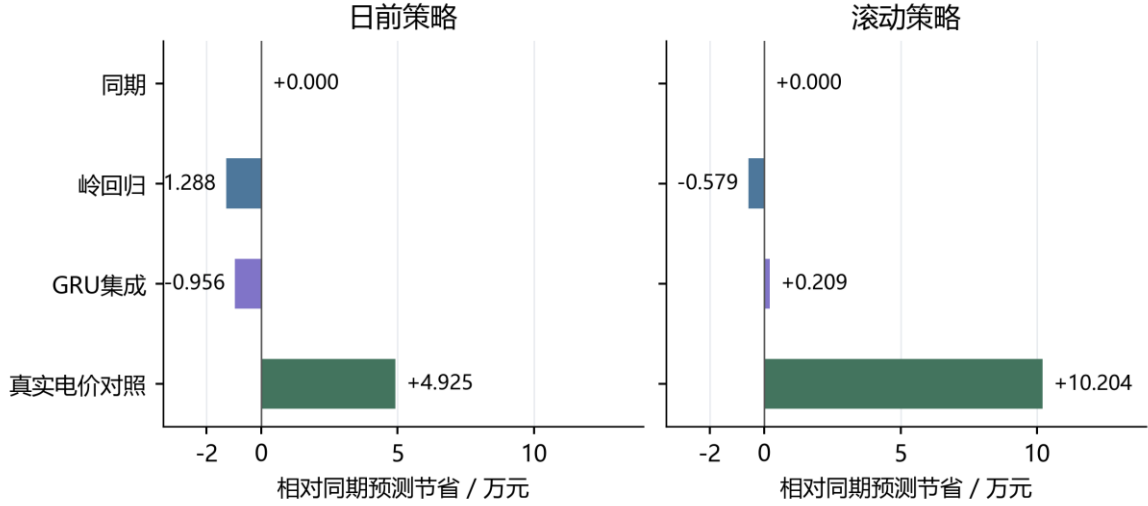


图 14 价格预测器对实际购电费用的影响

价格 MAE 改善没有按相同比例转化为电费下降。储能功率、容量和高低价次序限制了可改变的动作，部分预测改进发生在计划不变的区间。GRU 在本组滚动比较中费用最低；其相对同期的配对日节省在 7、14、28 天区块重采样下均有跨零区间，因此该小幅费用优势体现为本年度实测收益，而主要、稳定的证据是四个发布时刻一致的精度改善。结果文件采用一月选定的主策略，GRU 完整轨迹另随支撑材料保存。

9.5 名义最优下界与剩余改进上限

为直接回答价格预测还有多大优化空间，对每个日期固定相同的负载与光伏预测、风险余量、日初状态和日末目标，得到共同可行域 $\mathcal{F}_d$ 。各预测器只改变目标函数中的价格；事后将真实电价代入同一可行域求解线性规划[7]：

$$J_d^* = \min_{g \in \mathcal{F}_d} p_d^\top g, \quad J_{m,d} = p_d^\top g_{m,d}, \quad g_{m,d} \in \mathcal{F}_d. \quad (32)$$

由于真实电价解在相同可行域内最优，任一预测器的可行计划都满足 $J_{m,d} \geq J_d^*$ 。因此模型 m 仅通过改善价格预测能够获得的名义费用节省 Δm 受到下式约束：

$$0 \leq \Delta_m \leq R_m = \sum_d (J_{m,d} - J_d^*), \quad \gamma_m = \frac{R_m}{\sum_d J_{m,d}} \times 100\%. \quad (33)$$

对应结果见表 11。

表 11 相同名义可行域中的真实价格下界

预测器	名义费用/元	真实价格下界/元	剩余差额/元
同期	13,650,909.99	13,519,514.66	131,395.33
岭回归	13,663,504.91	13,519,514.66	143,990.25
GRU 集成	13,656,357.29	13,519,514.66	136,842.64

图 15 展示 334 个共同可行域上计算的最优值差距。该差额为相同名义调度问题内的改进上限，数值越小表示越接近真实价格最优计划。

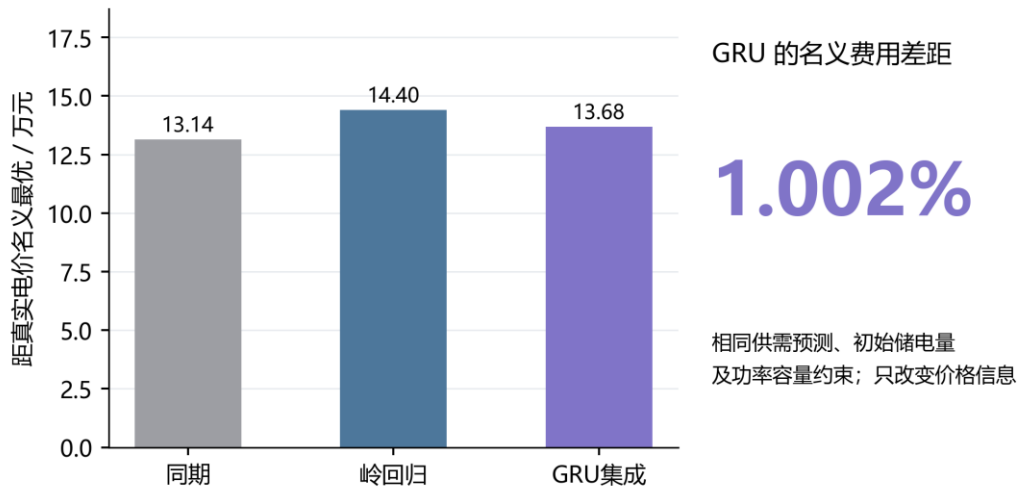


图 15 价格预测的名义费用改进空间

GRU 的名义计划费用为 13,656,357.29 元，真实电价最优下界为 13,519,514.66 元，两者相差 136,842.64 元，即 1.002%。这说明在既定供需预测和储能条件下，继续改进电价预测最多只能削减约 1% 的名义计划费用。同期与岭回归也处于相近量级，表明费用接近最优主要来自稳定的调度结构，而 GRU 的附加价值在于降低预测误差并改善部分滚动决策。

实际执行的真实电价对照另得：滚动费用为 14,212,407.27 元，相比同期预测改善 0.713%。该对照包含真实供需偏差与紧急购电，属于固定执行规则下的事后实验；上述约 1% 的数学上限则限定于共同名义可行域。两者共同指向：提高供需预报和调整策略的收益空间，比单独精细化电价预测更值得优先投入。

十 模型检验与评价

10.1 数值验证与结论稳健性

从原始附件到实际轨迹逐项检查日期、时段、单位和非有限值。第一问的 LP 与 MILP 费用一致；其余 16 组全年轨迹均满足供需平衡、储能递推、充放电互斥、功率容量边界和跨日连续性。费用复算独立采用“保留量+取消量+新增量+紧急量”公式，并从午夜计划逐次恢复最后有效计划，验证运行结果与费用结算一致。

时间因果性通过训练标签截止检查及未来输入扰动测试检验：改变未来实际数据，不应改变此前的预测与风险余量。原始观测未删点、截尾或平滑；价格预测输出的正值截断属于预测器约束。对 GRU 成本增益采用配对日区块重采样，保留日间相关性，其区间跨零；对所有方法，则通过共同名义可行域和最优下界验证价格优化空间。

10.2 模型的优点 局限与推广

模型的主要优点有三项。其一，利用循环消除证明将混合整数模型等价化为线性规划，保留物理可行性并降低求解复杂度。其二，把预测误差分位数与滚动结算共同纳入

调度，直接压缩高价补购而非只追求误差指标。其三，在价格预测之外给出费用最优下界，使神经网络的效果能够从精度、实际收益和剩余空间三个方面解释。

模型采用区间平均功率近似，实时层按即时平衡规则执行；电池退化与功率爬坡未纳入费用。年度数据及一月验证窗口限制了跨年推广，后续可增加气象变量、滚动交叉验证和电池循环成本，并将日末固定目标替换为跨日储能价值函数。加入线路与多节点约束后，同一费用结构可扩展至多个微网的协同购电。

十一 结论

四问结果汇总如下，各情景分别对应不同信息和计费条件。

对应结果见表 12。

表 12 四问模型与主策略结果

问题	模型及策略	总费用/元	评价区间
一	等价 LP 与周期储能	35,126.85	单日
二	岭回归+70%余量	14,132,612.16	334 天
三	滚动 LP+50%余量	13,575,976.78	334 天
四 日前	岭回归电价	14,880,535.05	334 天
四 滚动	同期电价与滚动 LP	14,314,442.48	334 天
四 GRU 对照	残差 GRU 与滚动 LP	14,312,347.70	334 天

储能使第一问节费 26.90%；第二问的风险余量在相同预测器下再降费 10.01%；第三问引入三次预报更新，较仅午夜预报降低 9.76%。第四问中，GRU 在四个发布时刻均提升预测精度，滚动费用为 1,431.23 万元；其名义计划费用距真实电价最优下界仅 1.002%。由此可见，风险校准与日内信息更新决定主要经济收益，价格算法的作用是进一步细化已接近最优的购电决策。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题 微网与外部电网电力调控策略及附件.
- [2] SciPy Developers. `scipy.optimize.linprog` and `scipy.optimize.milp`.
<https://docs.scipy.org/doc/scipy/reference/optimize.html>. 访问日期 2026-09-11.
- [3] scikit-learn Developers. Ridge. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html. 访问日期 2026-09-11.
- [4] Morstyn T, Hredzak B, Aguilera R P, Agelidis V G. Model Predictive Control for Distributed Microgrid Battery Energy Storage Systems. *IEEE Transactions on Control Systems Technology*, 2018, 26(3):1107–1114. DOI:10.1109/TCST.2017.2699159.
- [5] PyTorch Contributors. GRU. <https://docs.pytorch.org/docs/2.14/generated/torch.nn.GRU.html>. 访问日期 2026-09-11.
- [6] Cho K, van Merriënboer B, Gulcehre C, et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP*, 2014. arXiv:1406.1078.
- [7] Boyd S, Vandenberghe L. *Convex Optimization*. Cambridge University Press, 2004.
<https://web.stanford.edu/~boyd/cvxbook/>.

附录 A 四问指定日期完整结果

电量单位为 kWh，费用单位为元。普通购电与紧急购电分列。滚动问题的指定时段和全天购电量指最终有效普通计划；午夜计划及每次更新完整保存在 Excel 中。所有表由实际轨迹自动生成。

A.1 问题 1

对应结果见表 13。

表 13 问题 1（单日）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量 59,482.70 kWh			全天购电费 35,126.85 元		

对应结果见表 14。

表 14 问题 1（单日）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	5,947.42
08:00-12:00	3,954.63	2,121.42	12:00-16:00	6,119.37	91.10
16:00-20:00	0.00	5,068.69	20:00-24:00	5,333.33	3,571.31
00:00 储电	6,000.00		24:00 储电	6,000.00	

A.2 问题 2

对应结果见表 15。

表 15 问题 2（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	720.76	20:00-20:10	0.00
全天购电量 68,036.47 kWh			全天购电费 41,276.85 元		

对应结果见表 16。

表 16 问题 2（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,071.86	209.79	04:00-08:00	1,038.99	5,515.86
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	266.98	5,659.90	20:00-24:00	5,687.88	2,954.47
00:00 储电	6,617.05		24:00 储电	6,376.92	

对应结果见表 17。

表 17 问题 2（2025-03-20）紧急购电

时段	电量
无	0.00

对应结果见表 18。

表 18 问题 2（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,315.40 kWh			全天购电费 21,120.39 元		

对应结果见表 19。

表 19 问题 2（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	526.76	524.94	04:00-08:00	413.77	4,402.74
08:00-12:00	9,426.35	0.00	12:00-16:00	0.00	0.00
16:00-20:00	151.96	5,493.83	20:00-24:00	6,132.83	2,497.81
00:00 储电	6,945.01		24:00 储电	7,576.71	

对应结果见表 20。

表 20 问题 2（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 21。

表 21 问题 2（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	788.73	20:00-20:10	0.00
全天购电量 71,561.09 kWh			全天购电费 44,415.98 元		

对应结果见表 22。

表 22 问题 2（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,252.67	0.00	04:00-08:00	218.14	5,830.96
08:00-12:00	5,558.85	1,896.62	12:00-16:00	4,033.40	575.17
16:00-20:00	231.66	5,642.21	20:00-24:00	6,050.86	2,841.96
00:00 储电	6,072.60		24:00 储电	6,631.50	

对应结果见表 23。

表 23 问题 2（2025-09-23）紧急购电

时段	电量
20:20-20:30	0.33

对应结果见表 24。

表 24 问题 2（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,145.51 kWh			全天购电费 62,434.14 元		

对应结果见表 25。

表 25 问题 2（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,629.25	143.74	04:00-08:00	1,398.62	1,538.91
08:00-12:00	5,704.82	7,168.67	12:00-16:00	6,940.07	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,695.76	2,842.53
00:00 储电	6,318.60		24:00 储电	6,443.34	

对应结果见表 26。

表 26 问题 2（2025-12-21）紧急购电

时段	电量
07:50-08:00	23.78

A.3 问题 3

对应结果见表 27。

表 27 问题 3（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	518.93	14:00-14:10	0.00
16:00-16:10	638.18	18:00-18:10	714.94	20:00-20:10	0.00
全天购电量 66,321.27 kWh			全天购电费 43,773.17 元		

对应结果见表 28。

表 28 问题 3（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,249.82	348.06	04:00-08:00	985.61	5,965.48
08:00-12:00	3,123.27	2,523.36	12:00-16:00	6,532.46	254.72
16:00-20:00	674.89	5,121.85	20:00-24:00	5,562.08	3,004.05
00:00 储电	6,306.89		24:00 储电	6,191.65	

对应结果见表 29。

表 29 问题 3（2025-03-20） 紧急购电

时段	电量
09:10-10:00	254.02

对应结果见表 30。

表 30 问题 3（2025-06-21） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	117.96	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,802.44 kWh			全天购电费 19,709.89 元		

对应结果见表 31。

表 31 问题 3（2025-06-21） 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	275.76	271.67	04:00-08:00	436.62	4,038.73
08:00-12:00	9,885.67	0.00	12:00-16:00	0.00	0.00
16:00-20:00	57.45	5,815.05	20:00-24:00	5,796.74	2,550.02
00:00 储电	6,051.09		24:00 储电	6,774.25	

对应结果见表 32。

表 32 问题 3（2025-06-21） 紧急购电

时段	电量
无	0.00

对应结果见表 33。

表 33 问题 3（2025-09-23） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	347.56	14:00-14:10	0.00
16:00-16:10	518.73	18:00-18:10	779.85	20:00-20:10	0.00
全天购电量 68,335.67 kWh			全天购电费 44,072.98 元		

对应结果见表 34。

表 34 问题 3（2025-09-23） 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,292.60	16.12	04:00-08:00	395.29	6,697.97
08:00-12:00	5,033.01	1,896.62	12:00-16:00	5,842.71	515.80
16:00-20:00	100.73	5,868.44	20:00-24:00	5,726.09	2,611.46
00:00 储电	5,736.51		24:00 储电	6,325.22	

对应结果见表 35。

表 35 问题 3（2025-09-23）紧急购电

时段	电量
20:20-20:40	19.24
20:50-21:10	61.36
21:20-21:40	31.77

对应结果见表 36。

表 36 问题 3（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,721.86 kWh			全天购电费 62,219.35 元		

对应结果见表 37。

表 37 问题 3（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,657.30	179.36	04:00-08:00	1,565.97	1,572.10
08:00-12:00	5,310.40	7,513.79	12:00-16:00	7,773.62	2,250.94
16:00-20:00	57.98	5,766.67	20:00-24:00	5,676.93	2,842.53
00:00 储电	6,219.20		24:00 储电	6,395.64	

对应结果见表 38。

表 38 问题 3（2025-12-21）紧急购电

时段	电量
07:50-08:00	33.89

A.4-2 问题 4-2

对应结果见表 39。

表 39 问题 4-2（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	310.56	20:00-20:10	756.24
全天购电量 67,791.08 kWh			全天购电费 42,355.14 元		

对应结果见表 40。

表 40 问题 4-2（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,655.77	152.63	04:00-08:00	1,195.22	5,572.84
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	214.58	7,108.02	20:00-24:00	5,717.23	1,458.76
00:00 储电	6,850.72		24:00 储电	6,409.05	

对应结果见表 41。

表 41 问题 4-2（2025-03-20）紧急购电

时段	电量
无	0.00

对应结果见表 42。

表 42 问题 4-2（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,422.89 kWh			全天购电费 18,518.97 元		

对应结果见表 43。

表 43 问题 4-2（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	438.52	3,694.59	04:00-08:00	1,239.76	1,877.96
08:00-12:00	9,421.97	0.00	12:00-16:00	0.00	0.00
16:00-20:00	163.47	5,505.34	20:00-24:00	6,196.26	2,497.81
00:00 储电	7,001.50		24:00 储电	7,631.37	

对应结果见表 44。

表 44 问题 4-2（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 45。

表 45 问题 4-2（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	416.70	20:00-20:10	0.00
全天购电量 71,436.45 kWh			全天购电费 46,451.24 元		

对应结果见表 46。

表 46 问题 4-2（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,585.50	0.00	04:00-08:00	760.67	5,830.96
08:00-12:00	5,687.53	1,896.62	12:00-16:00	3,858.02	528.46
16:00-20:00	253.12	6,023.40	20:00-24:00	6,035.33	2,472.31
00:00 储电	6,184.77		24:00 储电	6,633.87	

对应结果见表 47。

表 47 问题 4-2（2025-09-23）紧急购电

时段	电量
20:10-20:20	10.23

对应结果见表 48。

表 48 问题 4-2（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,314.24 kWh			全天购电费 73,263.64 元		

对应结果见表 49。

表 49 问题 4-2（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,056.07	143.74	04:00-08:00	1,743.90	2,155.89
08:00-12:00	5,755.95	7,219.80	12:00-16:00	6,952.06	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,755.02	2,845.49
00:00 储电	6,309.24		24:00 储电	6,493.37	

对应结果见表 50。

表 50 问题 4-2（2025-12-21）紧急购电

时段	电量
07:50-08:00	23.78

A.4-3 问题 4-3

对应结果见表 51。

表 51 问题 4-3（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	518.93	14:00-14:10	0.00
16:00-16:10	638.18	18:00-18:10	714.94	20:00-20:10	741.44
全天购电量 66,016.42 kWh			全天购电费 44,515.75 元		

对应结果见表 52。

表 52 问题 4-3（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,264.78	249.34	04:00-08:00	1,804.98	6,081.65
08:00-12:00	3,123.27	2,525.35	12:00-16:00	6,961.11	254.72
16:00-20:00	292.25	6,623.44	20:00-24:00	5,594.46	1,541.15
00:00 储电	6,477.58		24:00 储电	6,219.19	

对应结果见表 53。

表 53 问题 4-3（2025-03-20）紧急购电

时段	电量
09:10-10:00	252.03

对应结果见表 54。

表 54 问题 4-3（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	117.96	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,875.45 kWh			全天购电费 17,362.67 元		

对应结果见表 55。

表 55 问题 4-3（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	248.99	3,229.59	04:00-08:00	1,265.39	1,786.95
08:00-12:00	9,917.23	0.00	12:00-16:00	0.00	0.00
16:00-20:00	57.45	5,815.05	20:00-24:00	5,858.38	2,548.85
00:00 储电	6,085.48		24:00 储电	6,831.03	

对应结果见表 56。

表 56 问题 4-3（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 57。

表 57 问题 4-3（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	347.56	14:00-14:10	0.00
16:00-16:10	518.73	18:00-18:10	607.08	20:00-20:10	0.00
全天购电量 68,304.99 kWh			全天购电费 46,106.59 元		

对应结果见表 58。

表 58 问题 4-3（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,679.06	22.22	04:00-08:00	1,026.50	6,697.97
08:00-12:00	5,033.01	1,896.62	12:00-16:00	5,821.44	467.89
16:00-20:00	95.08	6,023.69	20:00-24:00	5,575.37	2,482.31
00:00 储电	5,727.38		24:00 储电	6,189.57	

对应结果见表 59。

表 59 问题 4-3（2025-09-23）紧急购电

时段	电量
18:20-18:30	1.08
20:10-20:20	10.23
20:30-20:40	16.99
20:50-21:10	61.36
21:20-21:40	31.77

对应结果见表 60。

表 60 问题 4-3（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,641.71 kWh			全天购电费 72,890.05 元		

对应结果见表 61。

表 61 问题 4-3（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,169.62	178.51	04:00-08:00	232.98	904.57
08:00-12:00	5,347.35	7,538.30	12:00-16:00	8,080.06	2,504.57
16:00-20:00	57.98	5,766.67	20:00-24:00	5,708.45	2,845.49
00:00 储电	6,215.16		24:00 储电	6,420.72	

对应结果见表 62。

表 62 问题 4-3（2025-12-21）紧急购电

时段	电量
07:50-08:00	33.89

附录 B 支撑文件与完整程序

五份结果文件 result1、result2、result3、result4-2、result4-3 保存购电、储能和紧急购电结果。逐时轨迹 CSV、计划版本 CSV、价格预测权重与训练日志、名义价格下界表和 PNG/SVG 图形随支撑材料提供。原始题目附件单独读取。

代码采用 Python、NumPy、SciPy、pandas 和 PyTorch；绘图使用 Matplotlib。src/q12.py 负责基础数据与日前策略，src/q34_data.py 处理小时预报，src/q4_forecast.py 训练价格预测器；src/v3_q1.py 求解周期第一问，src/v3_model.py 实现最终合同结算与滚动 LP，src/v3_experiments.py 比较全年策略和名义价格下界，src/v3_verify.py 复算轨迹，src/v3_payload.py 及 scripts/export_v3_xlsx.mjs 导出结果。下列附录给出全部核心计算程序。

src/q12.py

"""Causal forecasts, MILP day-ahead dispatch and independent physical audits.

Run from the repository: `python -m src.q12 --data-root <C 題 directory> --stage full`
All power observations are interpreted as interval means at right-end labels.

"""

```
from __future__ import annotations
```

```
import argparse
from dataclasses import asdict, dataclass
import hashlib
import json
import os
from pathlib import Path
import platform
import subprocess
import time

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
import numpy as np
import pandas as pd
import scipy
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import lil_matrix
```

```
@dataclass(frozen=True)
class Battery:
    minimum: float = 1200.0
    maximum: float = 10800.0
    initial: float = 6000.0
    max_power: float = 5000.0
    eta_c: float = 0.9
    eta_d: float = 0.9
    dt: float = 1 / 6

    @property
    def limit(self):
        return self.max_power * self.dt
```

```
B = Battery()
N = 144
TOL = 1e-5
```

```
def dump_json(path, obj):
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
```

```
def interval_label(i):
    def clock(m):
        return f"{m // 60:02d}:{m % 60:02d}"
    return f"{clock(i * 10)}-{clock((i + 1) * 10)}"
```

```
def time_minutes(value):
    if hasattr(value, "hour"):
        return value.hour*60 + value.minute
    text = str(value).strip()
    next_day = "+1" in text
    hour, minute, *_ = text.replace("+1", "").split(":")
    return int(hour)*60 + int(minute) + 1440*int(next_day)
```

```
def read_inputs(root):
    root = Path(root)
```

```

day = pd.read_excel(root / "附件/附件 1.xlsx", sheet_name=0, header=0)
load = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="小区负载", header=0)
pv = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="光伏发电实际功率", header=0)
dates = pd.to_datetime(load.iloc[:, 0])
if day.shape != (144, 4) or load.shape != (365, 145) or pv.shape != (365, 145):
    raise ValueError("Unexpected input dimensions")
if not dates.equals(pd.to_datetime(pv.iloc[:, 0])):
    raise ValueError("Load/PV dates differ")
if list(dates) != list(pd.date_range("2025-01-01", "2025-12-31")):
    raise ValueError("Dates must cover the complete ordered year")
expected_minutes = list(range(10, 1441, 10))
if any([time_minutes(v) for v in labels] != expected_minutes
        for labels in (load.columns[1:], pv.columns[1:], day.iloc[:, 0])):
    raise ValueError("Input time labels do not match")
data = {
    "dates": pd.DatetimeIndex(dates), "price": day.iloc[:, 1].to_numpy(float),
    "q1_load": day.iloc[:, 2].to_numpy(float) * B.dt,
    "q1_pv": day.iloc[:, 3].to_numpy(float) * B.dt,
    "load": load.iloc[:, 1:].to_numpy(float) * B.dt,
    "pv": pv.iloc[:, 1:].to_numpy(float) * B.dt,
}
audit = {"shapes": {"attachment1": list(day.shape), "load": list(load.shape), "pv": list(pv.shape)},
        "date_start": str(dates.iloc[0].date()), "date_end": str(dates.iloc[-1].date()),
        "duplicates": int(dates.duplicated().sum()), "input_files": [], "variables": {}}
for key in ["price", "q1_load", "q1_pv", "load", "pv"]:
    a = data[key]
    if not np.isfinite(a).all() or (a < 0).any():
        raise ValueError(f"Nonfinite or negative data in {key}; investigate, do not silently impute")
    audit["variables"][key] = {"count": int(a.size), "missing": 0, "min": float(a.min()),
                              "max": float(a.max()), "zero_count": int((a == 0).sum())}
if (data["price"] <= 0).any():
    raise ValueError("This implementation assumes strictly positive fixed prices")
for relative in ["C 题.pdf", "附件/附件 1.xlsx", "附件/附件 2.xlsx"]:
    raw = (root / relative).read_bytes()
    audit["input_files"].append({"path": relative, "bytes": len(raw), "sha256":
hashlib.sha256(raw).hexdigest()})
    audit["data_treatment"] = "No missing/negative values found; no observations deleted or imputed. Unusual
peaks retained."
return data, audit

```

```

def optimize_day(load, pv, price, initial, terminal=6000.0, battery=B, integer=True):
    """Nominal minimum-cost schedule, g/c/d/w/S/z in kWh; terminal is a lower bound.

```

For Q1 initial=terminal and terminal_equal=True is obtained by the explicit upper bound below when requested by a (value, 'equal') tuple.

```

"""
n = len(price)
eq_terminal = isinstance(terminal, tuple)
target = terminal[0] if eq_terminal else terminal
size = 6 * n + 1
gi, ci, di, wi, si, zi = 0, n, 2*n, 3*n, 4*n, 5*n+1
objective = np.zeros(size)
objective[:n] = price
lb, ub = np.zeros(size), np.full(size, np.inf)
ub[ci:wi] = battery.limit
lb[si:zi], ub[si:zi] = battery.minimum, battery.maximum
lb[si] = ub[si] = initial
lb[si+n] = target
if eq_terminal:
    ub[si+n] = target
ub[zi:] = 1
integrality = np.zeros(size)
integrality[zi:] = int(integer)
a = lil_matrix((4*n, size))
lower, upper = np.zeros(4*n), np.zeros(4*n)
for t in range(n):
    a[t, gi+t], a[t, ci+t], a[t, di+t], a[t, wi+t] = 1, -1, 1, -1
    lower[t] = upper[t] = load[t] - pv[t]
    a[n+t, si+t+1], a[n+t, si+t] = 1, -1
    a[n+t, ci+t], a[n+t, di+t] = -battery.eta_c, 1 / battery.eta_d
    a[2*n+t, ci+t], a[2*n+t, zi+t] = 1, -battery.limit
    lower[2*n+t], upper[2*n+t] = -np.inf, 0
    a[3*n+t, di+t], a[3*n+t, zi+t] = 1, battery.limit
    lower[3*n+t], upper[3*n+t] = -np.inf, battery.limit
start = time.perf_counter()
result = milp(objective, integrality=integrality, bounds=Bounds(lb, ub),

```

```

        constraints=LinearConstraint(a.tocsc(), lower, upper),
        options={"time_limit": 30.0, "mip_rel_gap": 1e-7})
elapsed = time.perf_counter() - start
if not result.success or result.x is None:
    raise RuntimeError(f"MILP failed: status={result.status}, {result.message}")
x = result.x
sol = {"plan_kwh": x[:n], "charge_kwh": x[ci:di], "discharge_kwh": x[di:wi],
       "spill_kwh": x[wi:si], "soc_start_kwh": x[si:si+n], "soc_end_kwh": x[si+1:zi],
       "emergency_kwh": np.zeros(n), "load_kwh": np.asarray(load), "pv_kwh": np.asarray(pv)}
frame = pd.DataFrame(sol)
frame["price"] = price
meta = {"status": int(result.status), "cost": float(np.dot(price, x[:n])),
        "seconds": elapsed, "mip_gap": float(getattr(result, "mip_gap", 0) or 0),
        "dual_bound": float(getattr(result, "mip_dual_bound", result.fun) or result.fun)}
if integer:
    audit_trace(frame, battery=battery, initial=initial)
return frame, meta

def execute_day(plan, load, pv, initial, price, battery=B):
    """Causal real-time balancing; does not read future observations.

    Assumes instantaneous regulation within a ten-minute constant-power interval.
    Surplus charges first; deficit exhausts feasible discharge before emergency.
    """
    s = float(initial)
    records = []
    for t, (g, l, v) in enumerate(zip(plan, load, pv)):
        surplus = float(g + v - l)
        c = min(max(surplus, 0), battery.limit, max(0, (battery.maximum-s)/battery.eta_c))
        d = min(max(-surplus, 0), battery.limit, max(0, (s-battery.minimum)*battery.eta_d))
        e = max(0, -surplus-d)
        w = max(0, surplus-c)
        end = s + battery.eta_c*c - d/battery.eta_d
        records.append((g, l, v, c, d, e, w, s, end, price[t]))
        s = end
    frame = pd.DataFrame(records, columns=["plan_kwh", "load_kwh", "pv_kwh", "charge_kwh", "discharge_kwh",
                                          "emergency_kwh", "spill_kwh", "soc_start_kwh", "soc_end_kwh",
                                          "price"], dtype=float)
    audit_trace(frame, battery=battery, initial=initial)
    return frame

def audit_trace(frame, battery=B, initial=None):
    """Independent vectorized equations, also applicable to saved full-year traces."""
    f = frame
    balance = f.plan_kwh + f.pv_kwh + f.discharge_kwh + f.emergency_kwh - f.load_kwh - f.charge_kwh -
    f.spill_kwh
    evolution = f.soc_end_kwh - f.soc_start_kwh - battery.eta_c*f.charge_kwh + f.discharge_kwh/battery.eta_d
    continuity = f.soc_start_kwh.to_numpy()[1:] - f.soc_end_kwh.to_numpy()[:-1]
    metrics = {"max_balance_residual_kwh": float(np.max(np.abs(balance))),
              "max_soc_residual_kwh": float(np.max(np.abs(evolution))),
              "max_soc_discontinuity_kwh": float(np.max(np.abs(continuity))) if len(f)>1 else 0,
              "simultaneous_intervals": int(((f.charge_kwh > TOL) & (f.discharge_kwh > TOL)).sum()),
              "min_soc_kwh": float(min(f.soc_start_kwh.min(), f.soc_end_kwh.min())),
              "max_soc_kwh": float(max(f.soc_start_kwh.max(), f.soc_end_kwh.max()))}
    checks = [np.isfinite(f.select_dtypes(include="number").to_numpy()).all(),
              (f[["plan_kwh", "charge_kwh", "discharge_kwh", "emergency_kwh", "spill_kwh"]].to_numpy() >= -
    TOL).all(),
              metrics["max_balance_residual_kwh"] < TOL, metrics["max_soc_residual_kwh"] < TOL,
              metrics["max_soc_discontinuity_kwh"] < TOL, metrics["simultaneous_intervals"] == 0,
              metrics["min_soc_kwh"] >= battery.minimum-TOL, metrics["max_soc_kwh"] <= battery.maximum+TOL,
              f.charge_kwh.max() <= battery.limit+TOL, f.discharge_kwh.max() <= battery.limit+TOL]
    if initial is not None:
        checks.append(abs(f.soc_start_kwh.iloc[0]-initial) < TOL)
    if not all(checks):
        raise AssertionError(f"Physical audit failed: {metrics}")
    return {**metrics, "pass": True}

def features(history, day, dates):
    """Features for all 144 targets on a day, using completed days only."""
    if day < 7:
        raise ValueError("Seven completed days required")
    past = history[day-7:day]

```

```

h = (np.arange(N)+0.5)/N
dow = dates[day].dayofweek
cols = [past[-1], past[-2], past[0], past[-3:].mean(0), past.mean(0), past.std(0),
        np.full(N, past[-1].mean()), np.full(N, past[-3:].mean())]
for k in (1, 2, 3):
    cols.extend([np.sin(2*np.pi*k*h), np.cos(2*np.pi*k*h)])
cols.extend([np.full(N, float(dow == k)) for k in range(7)])
return np.column_stack(cols)

def ridge_fit(x, y, alpha):
    """Centered ridge via linear solve; intercept unpenalized, no sklearn required."""
    mean, scale = x.mean(0), x.std(0)
    scale[scale < 1e-10] = 1
    z = (x-mean)/scale
    intercept = float(y.mean())
    coef = np.linalg.solve(z.T@z + alpha*np.eye(z.shape[1]), z.T@(y-intercept))
    return mean, scale, coef, intercept

def forecasts(data, method, alphas=(10.0, 10.0), stop=365):
    pred, fit_dates = {}, []
    for k, alpha in zip(("load", "pv"), alphas):
        values = data[k]
        out = np.zeros((stop, N))
        model, fitted_on = None, -1
        all_x = {d: features(values, d, data["dates"]) for d in range(7, stop)}
        for d in range(stop):
            if d == 0:
                out[d] = data[f"q1_{k}"]
            elif d < 7:
                out[d] = values[max(0,d-3):d].mean(0)
            elif method == "seasonal" or d < 14:
                out[d] = 0.5*values[d-1] + 0.5*values[d-7]
            else:
                if model is None or d-fitted_on >= 7:
                    training_days = range(max(7,d-60), d)
                    x = np.concatenate([all_x[j] for j in training_days])
                    y = values[max(7,d-60):d].ravel()
                    model = ridge_fit(x, y, alpha)
                    fitted_on = d
                fit_dates.append({"variable": k, "decision_date": str(data["dates"][d].date()),
                                "fit_until": str(data["dates"][d-1].date()), "alpha": alpha,
                                "training_rows": len(y)})
                mean, scale, coef, intercept = model
                out[d] = ((all_x[d]-mean)/scale)@coef+intercept
            out[d] = np.maximum(out[d], 0)
            if k == "pv" and d:
                out[d, values[max(0,d-7):d].max(0) == 0] = 0
        pred[k] = out
    return pred, fit_dates

def reserve(data, forecast, day, quantile):
    if quantile == 0 or day < 7:
        return np.zeros(N)
    start = max(1, day-28)
    errors = (data["load"][start:day]-data["pv"][start:day]
              -forecast["load"][start:day]+forecast["pv"][start:day])
    # Pool +/- three slots; no wrap at the day boundary, no future residuals.
    return np.array([max(0.0, float(np.quantile(errors[:,max(0,t-3):min(N,t+4)], quantile))) for t in
                    range(N)])

def summarize_day(f, date, name, seconds, gap=0.0):
    normal = float(np.dot(f.plan_kwh, f.price))
    emergency = float(5*np.dot(f.emergency_kwh, f.price))
    return {"date": str(date.date()), "strategy": name, "normal_cost_yuan": normal,
            "emergency_cost_yuan": emergency, "total_cost_yuan": normal+emergency,
            "plan_kwh": float(f.plan_kwh.sum()), "emergency_kwh": float(f.emergency_kwh.sum()),
            "emergency_intervals": int((f.emergency_kwh>TOL).sum()), "spill_kwh": float(f.spill_kwh.sum()),
            "charge_kwh": float(f.charge_kwh.sum()), "discharge_kwh": float(f.discharge_kwh.sum()),
            "soc_start_kwh": float(f.soc_start_kwh.iloc[0]), "soc_end_kwh": float(f.soc_end_kwh.iloc[-1]),
            "solve_seconds": seconds, "mip_gap": gap}

```

```

def backtest(data, forecast, start, stop, initial, quantile, name, retain=True):
    s, daily, frames = initial, [], []
    for d in range(start, stop):
        buffer = reserve(data, forecast, d, quantile)
        nominal, meta = optimize_day(forecast["load"][d]+buffer, forecast["pv"][d], data["price"], s)
        f = execute_day(nominal.plan_kwh.to_numpy(), data["load"][d], data["pv"][d], s, data["price"])
        daily.append(summarize_day(f, data["dates"][d], name, meta["seconds"], meta["mip_gap"]))
        if retain:
            f.insert(0, "interval_start", pd.date_range(data["dates"][d], periods=N, freq="10min"))
            f.insert(1, "date", str(data["dates"][d].date()))
            f.insert(2, "slot", np.arange(N))
            f["forecast_load_kwh"], f["forecast_pv_kwh"], f["reserve_kwh"] = forecast["load"][d],
forecast["pv"][d], buffer
            frames.append(f)
            s = float(f.soc_end_kwh.iloc[-1])
            if retain and (d == start or data["dates"][d].day == 1):
                print(f"{name}: {data['dates'][d].date()}, cumulative cost={sum(x['total_cost_yuan'] for x in
daily):.2f}", flush=True)
            combined = pd.concat(frames, ignore_index=True) if frames else None
            if combined is not None:
                audit_trace(combined, initial=initial)
            return pd.DataFrame(daily), combined, s

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-root", type=Path, required=True)
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--stage", choices=["q1", "smoke", "full"], default="full")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    start = time.perf_counter()
    data, input_audit = read_inputs(args.data_root)
    dump_json(args.out/"input_audit.json", input_audit)
    q1, q1_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"))
    lp, lp_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"),
integer=False)
    q1.insert(0, "slot", np.arange(N)); q1.insert(1, "interval", [interval_label(i) for i in range(N)])
    q1.to_csv(args.out/"q1_intervals.csv", index=False)
    no_storage_cost = float(np.dot(np.maximum(data["q1_load"]-data["q1_pv"], 0), data["price"]))
    q1_meta.update({"no_storage_cost": no_storage_cost, "saving_percent": 100*(no_storage_cost-
q1_meta["cost"])/no_storage_cost,
                    "lp_cost_lower_bound": lp_meta["cost"], "lp_gap_yuan": q1_meta["cost"]-lp_meta["cost"],
                    "audit": audit_trace(q1, initial=B.initial), "plan_kwh": float(q1.plan_kwh.sum())})
    dump_json(args.out/"q1_summary.json", q1_meta)
    print("Q1", json.dumps(q1_meta, ensure_ascii=False), flush=True)
    if args.stage == "q1":
        return
    seasonal, _ = forecasts(data, "seasonal")
    tuning = []
    for alpha in (1.0, 10.0, 100.0):
        pred, _ = forecasts(data, "ridge", (alpha, alpha), stop=31)
        row = {"alpha": alpha}
        for key in ("load", "pv"):
            row[key+"_mae_kw"] = float(np.abs(pred[key][21:31]-data[key][21:31]).mean())/B.dt)
        tuning.append(row)
    alpha_load = min(tuning, key=lambda r: r["load_mae_kw"])["alpha"]
    alpha_pv = min(tuning, key=lambda r: r["pv_mae_kw"])["alpha"]
    ridge, fit_log = forecasts(data, "ridge", (alpha_load, alpha_pv))
    dump_json(args.out/"forecast_fit_log.json", fit_log)
    pd.DataFrame(tuning).to_csv(args.out/"ridge_validation.csv", index=False)
    _, _, validation_soc = backtest(data, seasonal, 0, 21, B.initial, 0, "warmup", retain=False)
    warmup, _, evaluation_soc = backtest(data, seasonal, 21, 31, validation_soc, 0, "warmup", retain=False)
    warmup.to_csv(args.out/"warmup_last10days.csv", index=False)
    validation = []
    for method, pred in (("seasonal", seasonal), ("ridge", ridge)):
        for q in (0.0, 0.7, 0.8, 0.9):
            daily, _, _ = backtest(data, pred, 21, 31, validation_soc, q, method, retain=False)
            validation.append({"forecast": method, "quantile": q, "cost_yuan": float(daily.total_cost_yuan.sum()),
                              "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
                              "end_soc_kwh": float(daily.soc_end_kwh.iloc[-1])})
    selected = min(validation, key=lambda r: r["cost_yuan"])
    pd.DataFrame(validation).to_csv(args.out/"policy_validation.csv", index=False)
    stop = 38 if args.stage == "smoke" else 365
    strategies = [("seasonal_q0", "seasonal", 0.0), ("ridge_q0", "ridge", 0.0)]

```

```

for method in ("seasonal", "ridge"):
    best = min((v for v in validation if v["forecast"]==method), key=lambda r: r["cost_yuan"])
    if best["quantile"]:
        strategies.append((f"{method}_q{best['quantile']}:g", method, best["quantile"]))
selected_name = f"{selected['forecast']}_q{selected['quantile']}:g"
all_daily, metrics = [], []
for name, method, q in strategies:
    pred = {"seasonal": seasonal, "ridge": ridge}[method]
    daily, trace, _ = backtest(data, pred, 31, stop, evaluation_soc, q, name)
    all_daily.append(daily)
    trace.to_csv(args.out/f"{name}_intervals.csv.gz", index=False, float_format="%.17g")
    audit = audit_trace(trace, initial=evaluation_soc)
    row = {"strategy": name, "forecast": method, "quantile": q, "selected_on_january": name==selected_name,
           "days": len(daily), "total_cost_yuan": float(daily.total_cost_yuan.sum()),
           "normal_cost_yuan": float(daily.normal_cost_yuan.sum()),
           "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
           "emergency_kwh": float(daily.emergency_kwh.sum()), "emergency_intervals": int(daily.emergency_intervals.sum()),
           "plan_kwh": float(daily.plan_kwh.sum()), "spill_kwh": float(daily.spill_kwh.sum()),
           "solve_seconds": float(daily.solve_seconds.sum()), "initial_soc_kwh": evaluation_soc,
           "final_soc_kwh": float(daily.soc_end_kwh.iloc[-1]), "audit": audit}
    for key in ("load", "pv"):
        error = (pred[key][31:stop]-data[key][31:stop])/B.dt
        row[key+"_mae_kw"], row[key+"_rmse_kw"] = float(np.abs(error).mean()), float(np.sqrt(np.mean(error**2)))
    metrics.append(row)
    daily = pd.concat(all_daily, ignore_index=True)
    daily.to_csv(args.out/f"q2_daily_metrics.csv", index=False)

dump_json(args.out/"q2_summary.json", {"selection": selected, "selected_strategy": selected_name, "strategies": metrics})
command = f'python -m src.q12 --data-root "{args.data_root}" --out "{args.out}" --stage {args.stage}'
manifest = {"stage": args.stage, "battery": asdict(B), "time_interpretation": "interval mean power, right-end labels",
            "alpha_load": alpha_load, "alpha_pv": alpha_pv, "validation_start": "2025-01-22", "validation_end": "2025-01-31",
            "evaluation_start": "2025-02-01", "evaluation_end": str(data["dates"][stop-1].date()),
            "validation_initial_soc": validation_soc, "evaluation_initial_soc": evaluation_soc,
            "terminal_nominal_soc_minimum": 6000, "refit_period_days": 7, "training_window_days": 60,
            "residual_window_days": 28, "pool_halfwidth_slots": 3, "seed": 2026, "deterministic": True,
            "selected_strategy": selected_name, "command": command, "total_seconds": time.perf_counter()-start,
            "python": platform.python_version(), "numpy": np.__version__, "scipy": scipy.__version__, "pandas": pd.__version__,
            "platform": platform.platform(), "git_head_before_run": subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).strip(),
            "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(), "input_files": input_audit["input_files"]}
dump_json(args.out/"run_manifest.json", manifest)
print("COMPLETE", json.dumps({"selected": selected_name, "seconds": manifest["total_seconds"], "days": stop-31}), flush=True)

if __name__ == "__main__":
    main()

```

src/q34_data.py

```

"""Strict attachment 3/4 parsing and causal hourly forecast alignment."""
from pathlib import Path
import hashlib
import numpy as np
import pandas as pd
import openpyxl
from src.q12 import read_inputs, time_minutes, dump_json

def read_extended(root, audit_path=None):
    root=Path(root); data, base_audit=read_inputs(root)
    wb=openpyxl.load_workbook(root/'附件/附件 3.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==1461 and len(rows[0])==26
    assert list(rows[0][2:]) == [f'预报{i}小时' for i in range(1,25)]
    hourly=np.empty((365,4,24)); filled=0
    for day in range(365):
        date=data['dates'][day]
        for version in range(4):
            row=rows[1+day*4+version]
            if version==0:
                assert pd.Timestamp(row[0])==date
            elif row[0] in (None,''):
                filled+=1
            else: assert pd.Timestamp(row[0])==date
            assert time_minutes(row[1])==version*360
            hourly[day,version]=np.asarray(row[2:],float)
    wb=openpyxl.load_workbook(root/'附件/附件 4.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==366 and len(rows[0])==145
    assert [time_minutes(v) for v in rows[0][1:]]==list(range(10,1441,10))
    assert pd.DatetimeIndex([r[0] for r in rows[1:]]).equals(data['dates'])
    prices=np.asarray([r[1:] for r in rows[1:]],float)
    checks={}
    for name, values in [(('hourly_pv_kw', hourly), ('actual_price', prices))]:
        assert np.isfinite(values).all(), f'{name}: missing/nonfinite'
        assert (values==0).all(), f'{name}: negative values need investigation'
        if name=='actual_price': assert (values>0).all()
        q1,q3=np.quantile(values, [.25, .75]); iqr=q3-q1
        checks[name]={ 'shape': list(values.shape), 'count': values.size, 'missing_nonfinite': 0,
            'negative': 0, 'zero': int((values==0).sum()), 'min': float(values.min()), 'max': float(values.max()),
            'iqr_flags_retained': int(((values<q1-1.5*iqr)|(values>q3+1.5*iqr)).sum()),
            'deleted': 0, 'imputed_numeric': 0, 'winsorized': 0, 'smoothed_observations': 0 }
    # An hourly point is an instantaneous forecast. Integrate its piecewise-linear
    # curve exactly over 10-minute cells. At lead 0 use the newest completed PV
    # interval (causally known) as a boundary proxy; midnight Jan 1 uses zero.
    pv=np.full((365,4,144), np.nan)
    for day in range(365):
        for version in range(4):
            start=version*36
            anchor=(data['pv'][day,start-1]*6 if start else
                data['pv'][day-1,-1]*6 if day else 0.0)
            vals=np.r_[anchor, hourly[day,version]]
            edges=np.arange(145-start)/6
            powers=np.interp(edges, np.arange(25), vals)
            pv[day,version,start:]= (powers[:-1]+powers[1:])/12
    data.update(hourly_pv=hourly, pv_issued=pv, actual_price=prices)
    audit={'pass': True, 'checks': checks, 'date_forward_fills_structural_only': filled,
        'dates': 365, 'unique_issues': 1460, 'issue_hours': [0,6,12,18],
        'forecast_horizons_hours': list(range(1,25)),
        'alignment': 'Hourly lead endpoints; piecewise-linear integration over 10-minute cells; lead-zero
anchor from last completed observed PV interval; only current-day suffix traded.',
        'input_files': base_audit['input_files']+[
            {'path': name, 'sha256': hashlib.sha256((root/name).read_bytes()).hexdigest()}
            for name in ['附件/附件 3.xlsx', '附件/附件 4.xlsx']],
        'unusable_past_slots': 'NaN intentionally marks targets before each issue; never supplied to a
solver',
        'auditor_sha256': hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
    if audit_path: dump_json(audit_path, audit)
    return data, audit

if __name__ == '__main__':

```



```
import argparse, json
p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
p.add_argument('--out',default='artifacts/q34/input_audit.json');args=p.parse_args()
Path(args.out).parent.mkdir(parents=True,exist_ok=True)
_,audit=read_extended(args.data_root,args.out)
print(json.dumps(audit,ensure_ascii=False,indent=2))
```

src/q4_forecast.py

"""Direct 24-hour price forecasts at four issue times, with monthly causal fits.

Seasonal, ridge and GRU see the same day/week lag curves and calendar. GRU predicts a residual around the seasonal curve, not future observed prices.

```
"""
from pathlib import Path
import argparse
import copy
import hashlib
import json
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from src.q12 import ridge_fit, dump_json
from src.q34_data import read_extended

torch.set_num_threads(1)

def calendar(indices):
    hours=(indices%144+.5)/144
    dow=(indices//144+2)%7 # 2025-01-01 Wednesday
    return np.stack([np.sin(2*np.pi*hours),np.cos(2*np.pi*hours),
                     np.sin(2*np.pi*dow/7),np.cos(2*np.pi*dow/7)],axis=-1)

def features_at(flat,origin):
    assert origin>=1008
    target=np.arange(origin,origin+144)
    return np.column_stack([flat[target-144],flat[target-1008],calendar(target)]).astype('float32')

def ridge_features(x):
    a,b=x[...,:0],x[...,:1]
    context=np.stack([a.mean(-1),b.mean(-1),a.std(-1),b.std(-1)],axis=-1)
    context=np.broadcast_to(context[...,:None,:],(*a.shape,4))
    return np.concatenate([x,np.stack([a-b,a*b],axis=-1),context],axis=-1)

class PriceGRU(nn.Module):
    def __init__(self):
        super().__init__();self.gru=nn.GRU(6,32,batch_first=True);self.head=nn.Linear(32,1)
        nn.init.zeros_(self.head.weight);nn.init.zeros_(self.head.bias)
    def forward(self,x):
        h,_=self.gru(x)
        return .5*(x[:, :, :0]+x[:, :, :1])+self.head(h).squeeze(-1)

def train_gru(x,y,origins,cutoff,seed,max_epochs=50):
    train=(origins+144<=cutoff-3*144);valid=(origins>=cutoff-3*144)
    assert train.sum()>0 and valid.sum()>0
    torch.manual_seed(seed);torch.use_deterministic_algorithms(True)
    mean=float(x[train,:, :2].mean());scale=max(float(x[train,:, :2].std()),1e-6)
    z=x.copy();z[:, :, :2]=(z[:, :, :2]-mean)/scale;target=(y-mean)/scale

    tx=torch.tensor(z[train]);ty=torch.tensor(target[train]);vx=torch.tensor(z[valid]);vy=torch.tensor(target[valid])
    model=PriceGRU();opt=torch.optim.Adam(model.parameters(),lr=.003)
    best=float('inf');weights=None;stale=0;history=[];rng=np.random.default_rng(seed)
    for epoch in range(max_epochs):
        model.train();order=rng.permutation(len(tx));losses=[]
        for left in range(0,len(order),32):
            idx=order[left:left+32];opt.zero_grad()
            loss=nn.functional.smooth_l1_loss(model(tx[idx]),ty[idx])
        loss.backward();nn.utils.clip_grad_norm_(model.parameters(),1.0);opt.step();losses.append(float(loss.detach()))
    model.eval()
```

```

        with torch.no_grad():score=float((model(vx)-vy).abs().mean())*scale
    history.append({'epoch':epoch+1,'training_huber':float(np.mean(losses)), 'inner_validation_mae':score})
        if score<best-1e-7:best=score;weights=copy.deepcopy(model.state_dict());stale=0
        else:stale+=1
        if stale>=6:break
    model.load_state_dict(weights);model.eval()
    return model,mean,scale,history,int(train.sum()),int(valid.sum())

def same_day_errors(pred,actual,start,stop):
    values=[]
    for day in range(start,stop):
        for v in range(4):values.extend(pred[day,v,:144-v*36]-actual[day,v*36:])
    a=np.asarray(values)
    return {'mae':float(np.abs(a).mean()), 'rmse':float(np.sqrt(np.mean(a*a))), 'targets':len(a)}

def main():
    p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
    p.add_argument('--out',type=Path,default=Path('artifacts/q34'));p.add_argument('--
    smoke',action='store_true')

    args=p.parse_args();out=args.out;out.mkdir(parents=True,exist_ok=True);(out/'models').mkdir(exist_ok=True)
    data,_=read_extended(args.data_root);flat=data['actual_price'].ravel();started=time.perf_counter()
    stop=38 if args.smoke else 365
    origins=np.arange(1008,len(flat)-143,36)
    x=np.stack([features_at(flat,t) for t in origins]);y=np.stack([flat[t:t+144] for t in
    origins]).astype('float32')
    # Fit on Jan 15/22 then each month boundary; never use targets ending after fit time.
    fit_days=[d for d in range(14,stop) if d in [14,21] or data['dates'][d].day==1]
    result={name:np.full((stop,4,144),np.nan,dtype='float64') for name in
    ['seasonal','ridge1','ridge10','ridge100','gru17','gru42','gru2026']}
    all_logs=[]
    for day in range(7,stop):
        for v in range(4):
            sample=features_at(flat,day*144+v*36)
            result['seasonal'][day,v]=sample[:,2].mean(-1)
    for fi,day in enumerate(fit_days):
        cutoff=day*144;end=fit_days[fi+1] if fi+1<len(fit_days) else stop
        choose=(origins+144<=cutoff)&(origins>=max(1008,cutoff-60*144))
        xx=x[choose];yy=y[choose];oo=origins[choose]
        assert (oo+144<=cutoff).all()
        future_x=np.stack([features_at(flat,d*144+v*36) for d in range(day,end) for v in range(4)])
        for alpha in [1.,10.,100.]:
            a=ridge_features(xx).reshape(-1,12);b=yy.ravel()
            model=ridge_fit(a,b,alpha);mean,scale,coef,intercept=model
            forecasts=((ridge_features(future_x)-mean)/scale)@coef+intercept
            result[f'ridge{alpha:g}'][day:end]=np.maximum(forecasts.reshape(end-day,4,144),.001)
        for seed in [17,42,2026]:
            t=time.perf_counter();model,mean,scale,history,nt,nv=train_gru(xx,yy,oo,cutoff,seed+day,max_epochs=12 if
            args.smoke else 50)
            z=future_x.copy();z[:,2,:]=z[:,2,:]-mean/scale
            with torch.no_grad():values=model(torch.tensor(z)).numpy()*scale+mean
            result[f'gru{seed}'][day:end]=np.maximum(values.reshape(end-day,4,144),.001)
            checkpoint=out/'models'/f'gru{seed}_fit{day:03d}.pt'
            torch.save({'state_dict':model.state_dict(),'mean':mean,'scale':scale,'fit_day':day,'seed':seed,
            'training_last_target_exclusive':cutoff,'architecture':'GRU(6,32)+Linear(32,1)+seasonal
            residual'},checkpoint)

    log={'fit_day':day,'fit_date':str(data['dates'][day].date()),'seed':seed,'actual_rng_seed':seed+day,
    'training_windows':nt,'inner_validation_windows':nv,'last_target_exclusive':int((oo+144).max()),
    'seconds':time.perf_counter()-t,'history':history,'checkpoint':str(checkpoint)}
    all_logs.append(log)
    print(f'price GRU seed={seed} fit={day}: {len(history)} epochs, {log["seconds"]:.1f}s, best
    inner MAE={min(r["inner_validation_mae"] for r in history):.4f}',flush=True)
    np.savez_compressed(out/'price_forecasts_checkpoint.npz',**result)
    dump_json(out/'price_fit_log.json',all_logs)
    for name in result:
        if name!='seasonal':result[name][7:14]=result['seasonal'][7:14]
        result['gru_mean']=np.mean([result[f'gru{s}'] for s in [17,42,2026]],axis=0)
        validation={name:same_day_errors(values,data['actual_price'],21,31) for name,values in result.items()}
        ridge_name=min(['ridge1','ridge10','ridge100'],key=lambda n:validation[n]['mae'])
        result['ridge']=result[ridge_name]
        chosen=min(['seasonal','ridge','gru_mean'],key=lambda n:validation[ridge_name if n=='ridge' else

```

```

n]['mae'])
    metrics={name:same_day_errors(result[name],data['actual_price'],31,stop) for name in
['seasonal','ridge','gru17','gru42','gru2026','gru_mean']}
    for name in ['seasonal','ridge','gru17','gru42','gru2026','gru_mean']:assert
np.isfinite(result[name][21:]).all()
    np.savez_compressed(out/'price_forecasts.npz',**result)

dump_json(out/'price_forecast_summary.json',{'validation':validation,'evaluation':metrics,'ridge_selected':r
idge_name,
        'selected_by_validation_price_mae':chosen,'seeds':[17,42,2026],'torch_version':torch.__version__,
        'epochs_max':12 if args.smoke else 50,'fit_days':fit_days,'elapsed_seconds':time.perf_counter()-
started,
        'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        'evaluation_note':'Same-day suffix only, at each issue time; repeated targets at different origins
counted separately.',
        'training_rule':'Whole 24h targets end before fit; last 3 days internal validation; refit monthly
from scratch, last 60 days; Jan15/22 initialization.',
        'clip_floor':.001,'architecture':'Direct aligned day/week lag sequences plus known calendar;
residual GRU(6,32), no future teacher forcing'})

if __name__=='__main__':main()

```

src/v3_model.py

```

"""Exact LP reformulation and causal forecast-corrected dispatch.

Energy in kWh. Same settlement, time axis and realtime regulation as V1.
No training or tuning is performed here; inputs explicitly carry decisions.
"""

from dataclasses import dataclass, asdict
from functools import lru_cache
import time
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from src.q12 import B, TOL, audit_trace, execute_day

@lru_cache(maxsize=32)
def structure(n, revision):
    # g,c,d,w,S[,increase,decrease]; state has n+1 boundary values.
    size = (7 if revision else 5)*n+1
    a = lil_matrix(((3 if revision else 2)*n, size))
    for t in range(n):
        a[t,t], a[t,n+t], a[t,2*n+t], a[t,3*n+t] = 1,-1,1,-1
        a[n+t,4*n+t+1], a[n+t,4*n+t] = 1,-1
        a[n+t,n+t], a[n+t,2*n+t] = -B.eta_c,1/B.eta_d
        if revision:
            a[2*n+t,t], a[2*n+t,5*n+1+t], a[2*n+t,6*n+1+t] = 1,-1,1
    return a.tocsr()

def remove_cycles(charge, discharge, spill):
    """Preserve grid purchases and ALL SOC values, eliminate simultaneous flow.

    x=min(c,d/(eta_c*eta_d)); c'=c-x; d'=d-eta_c*eta_d*x;
    w'=w+(1-eta_c*eta_d)*x. Requires free unbounded curtailment.
    """
    rho=B.eta_c*B.eta_d
    x=np.minimum(charge,discharge/rho)
    return charge-x,discharge-rho*x,spill+(1-rho)*x

def solve(load,pv,price,initial,terminal=6000.,old=None,refund=True,equal=False):
    load,pv,price=map(lambda x:np.asarray(x,dtype=float),(load,pv,price))
    n=len(price);revision=old is not None;size=(7 if revision else 5)*n+1
    if not all(x.shape==(n,) and np.isfinite(x).all() for x in (load,pv,price)):
        raise ValueError('Nonfinite or misaligned dispatch inputs')
    if np.any(price<=0) or not B.minimum<=terminal<=B.maximum:
        raise ValueError('Positive prices and admissible terminal storage required')
    bounds=[(0,None)]*size
    bounds[n:3*n]=[(0,B.limit)]*(2*n)
    bounds[4*n:5*n+1]=[(B.minimum,B.maximum)]*(n+1)
    bounds[4*n]=(initial,initial);bounds[5*n]=(terminal,terminal if equal else B.maximum)
    cost=np.zeros(size)
    rhs=np.r_[load-pv,np.zeros(n)]
    if revision:
        old=np.asarray(old,dtype=float)
        assert old.shape==(n,) and np.isfinite(old).all() and old.min()>=-TOL
        cost[5*n+1:6*n+1]=1.5*price
        cost[6*n+1:]=(.5 if refund else .5)*price
        bounds[6*n+1:]=[(0,max(float(v),0)) for v in old]
        rhs=np.r_[rhs,old]
    else:cost[:n]=price
    t=time.perf_counter()
    fit=linprog(cost,A_eq=structure(n,revision),b_eq=rhs,bounds=bounds,method='highs',
        options={'time_limit':30,'dual_feasibility_tolerance':1e-
8,'primal_feasibility_tolerance':1e-8})
    if not fit.success:raise RuntimeError(f'LP status={fit.status}: {fit.message}')
    x=fit.x;c,d,w=remove_cycles(x[n:2*n],x[2*n:3*n],x[3*n:4*n])
    f=pd.DataFrame(dict(plan_kwh=x[:n],charge_kwh=c,discharge_kwh=d,spill_kwh=w,
        soc_start_kwh=x[4*n:5*n],soc_end_kwh=x[4*n+1:5*n+1],load_kwh=load,pv_kwh=pv,
        emergency_kwh=np.zeros(n),price=price))
    physical=audit_trace(f,initial=initial)
    return f,dict(objective=float(fit.fun),seconds=time.perf_counter()-t,status=int(fit.status),

```

```

iterations=int(fit.nit),physical=physical)

@dataclass(frozen=True)
class Policy:
    quantile:float=.7
    terminal:float=6000.
    mask:int=0
    beta:float=0.
    refund:bool=True
    lookback:int=18
    decay:int=36

def corrected_load(data,pred,beta,lookback=18,decay=36):
    """At each release, use only the just-completed <=3h load residuals.

    No observed PV is used to correct load; no current/future interval enters.
    Correction decays with lead, preventing all-day propagation of a short shock.
    """
    out=np.repeat(pred['load'][:,None,:],4,axis=1).copy()
    for v in range(1,4):
        start=36*v;left=max(0,start-lookback)
        err=(data['load'][:,left:start]-pred['load'][:,left:start]).mean(axis=1)
        lead=np.arange(144-start)+.5
        out[:,v,start:]=np.maximum(0,out[:,v,start:]+beta*err[:,None]*np.exp(-lead[None,:]/decay))
    return out

def risk_buffers(data,load_pred,pv_pred,q):
    out=np.zeros_like(load_pred)
    if q==0:return out
    for day in range(7,len(load_pred)):
        left=max(1,day-28)
        error=data['load'][:,left:day,None,:]-data['pv'][:,left:day,None,:]-load_pred[left:day]+pv_pred[left:day]
        for v in range(4):
            start=36*v
            # Historical same-release residuals, not errors from unreleased versions.
            for slot in range(start,144):
                out[day,v,slot]=max(0,float(np.quantile(error[:,v,max(start,slot-3):min(144,slot+4)],q)))
    return out

def run(data,load_pred,pv_pred,buffers,policy,*,start,stop,initial,name,prices=None,retain=True):
    state=float(initial);frames=[];days=[];ledger=[];solves=[]
    for day in range(start,stop):
        date=str(data['dates'][day].date());s0=state
        realized=data['price'] if prices is None else data['actual_price'][day]
        p0=data['price'] if prices is None else prices[day,0,:144]
        sol,meta=solve(load_pred[day,0]+buffers[day,0],pv_pred[day,0],p0,state,policy.terminal)
        solves.append({'date':date,'version':0,**{k:meta[k] for k in
['seconds','status','iterations','objective']})})
        plan=np.maximum(sol.plan_kwh.to_numpy(),0);original=plan.copy()
        up=np.zeros(144);down=np.zeros(144);blocks=[]
        for v in range(4):
            first=v*36;last=first+36
            if v and policy.mask & (1<<(v-1)):
                p=data['price'][first:] if prices is None else prices[day,v,:144-first]
                old=plan[first:].copy()
                sol,meta=solve(load_pred[day,v,first:]+buffers[day,v,first:],pv_pred[day,v,first:],
                    p,state,policy.terminal,original[first:],policy.refund)
                solves.append({'date':date,'version':v,**{k:meta[k] for k in
['seconds','status','iterations','objective']})})
                new=np.maximum(sol.plan_kwh.to_numpy(),0);delta=new-old
                up[first:]+=np.maximum(delta,0);down[first:]+=np.maximum(-delta,0);plan[first:]=new
                if retain:
                    for j,slot in enumerate(range(first,144)):
                        ledger.append([date,v,first,slot,float(old[j]),float(new[j]),max(float(delta[j]),0),
                            max(-float(delta[j]),0),float(p[j]),float(load_pred[day,v,slot]),
                                float(pv_pred[day,v,slot]),float(buffers[day,v,slot]),state)])
    f=execute_day(plan[first:last],data['load'][day,first:last],data['pv'][day,first:last],state,realized[first:
last])
    state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)

```

```

f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',np.arange(144))
# Publication versions control execution; the final bill is relative to
# the midnight contract, not the sum of intermediate revision movements.
up=np.maximum(plan-original,0);down=np.maximum(original-plan,0)
f['original_plan_kwh']=original;f['increase_kwh']=up;f['decrease_kwh']=down
f['original_cost_yuan']=realized*original;f['increase_cost_yuan']=realized*1.5*up
f['decrease_cost_yuan']=realized*(-.5 if policy.refund else .5)*down
f['emergency_cost_yuan']=realized*5*f.emergency_kwh

f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
np.testing.assert_allclose(f.plan_kwh,original+up-down,atol=TOL,rtol=0)
item={'date':date,'strategy':name,'soc_start_kwh':s0,'soc_end_kwh':state}
for c in f:
    if c.endswith('_yuan') or c in ['original_plan_kwh','plan_kwh','increase_kwh','decrease_kwh',
        'emergency_kwh','spill_kwh','charge_kwh','discharge_kwh']:item[c]=float(f[c].sum())
    item['emergency_intervals']=int((f.emergency_kwh>TOL).sum());days.append(item)
    if retain:frames.append(f)
daily=pd.DataFrame(days);trace=pd.concat(frames,ignore_index=True) if retain else None
versions=pd.DataFrame(ledger,columns=['date','version','effective_slot','slot','old_kwh','new_kwh',
    'increase_kwh','decrease_kwh','forecast_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision'
    '_soc_kwh'])
summary={'strategy':name,'policy':asdict(policy),'days':stop-
start,'initial_soc':initial,'final_soc':state,
    'solver_seconds':sum(x['seconds'] for x in solves),'solver_calls':len(solves),
    'physical':audit_trace(trace,initial=initial) if retain else None,
    **{c:float(daily[c].sum()) for c in daily if c.endswith('_yuan') or c in
    ['emergency_kwh','spill_kwh']}}
return daily,trace,versions,summary,pd.DataFrame(solves)

```

src/v3_q1.py

```
"""Q1 template-start sampling with explicit periodic midnight closure."""
from pathlib import Path
from dataclasses import replace
import argparse
import numpy as np
import pandas as pd
from src.q12 import read_inputs, optimize_day, B, dump_json, interval_label, audit_trace
from src.v3_model import solve

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    data,_=read_inputs(a.data_root); out=Path('artifacts/v3'); out.mkdir(parents=True, exist_ok=True)
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    f,meta=solve(L,V,p,6000,equal=True);_,milp=optimize_day(L,V,p,6000,(6000,'equal'))
    assert abs(meta['objective']-milp['cost'])<1e-6
    f.insert(0,'slot',range(144)); f.insert(1,'interval',[interval_label(i) for i in range(144)])
    f.to_csv(out/'q1_intervals.csv',index=False,float_format='%.17g')
    sensitivity=[]
    for eta in [.8,.85,.9,np.sqrt(.9),1.]:
        b=replace(B,eta_c=eta,eta_d=eta); ff,mm=optimize_day(L,V,p,6000,(6000,'equal'),battery=b)
        audit_trace(ff,battery=b,initial=6000)
        sensitivity.append({'single_efficiency':eta,'roundtrip_efficiency':eta*eta,'cost':mm['cost']})
    pd.DataFrame(sensitivity).to_csv(out/'q1_efficiency.csv',index=False)
    dump_json(out/'q1_summary.json',{'cost':meta['objective'],'no_storage_cost':float(p@np.maximum(L-V,0)),
    'plan_kwh':float(f.plan_kwh.sum()),'charge_kwh':float(f.charge_kwh.sum()),'discharge_kwh':float(f.discharge_
    kwh.sum()),
    'milp':milp,'lp':meta,'specified':{interval_label(i):float(f.plan_kwh.iloc[i]) for i in
    [60,72,84,96,108,120]}},
    'alignment':'Template start timestamps; Q1 periodic 24:00 sample fills 00:00 and all signals are
    jointly rotated; solve again with actual 00:00=24:00 SOC=6000.',
    'historical_series':'Q2-Q4 retained interval-average/right-end convention; no same-day final sample
    copied into their midnight histories.')}
    print('Q1',meta['objective'],milp['cost'])

if __name__=='__main__':main()
```


src/v3_experiments.py

```

"""Final-versus-midnight settlement and paired price-information experiments."""
from pathlib import Path
import argparse, json, time, hashlib
import numpy as np
import pandas as pd
from src.q12 import forecasts, dump_json
from src.q34_data import read_extended
from src.v3_model import Policy, corrected_load, risk_buffers, run, solve

OUT=Path('artifacts/v3')
def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    OUT.mkdir(parents=True, exist_ok=True); beg=time.perf_counter()
    data, audit=read_extended(a.data_root); pred,_=forecasts(data, 'ridge', (1., 1.))
    pp=np.load('artifacts/q34/price_forecasts.npz'); ll=corrected_load(data, pred, 0)
    pv2=np.repeat(pred['pv'][:, None, :], 4, axis=1); pv3=data['pv_issued']
    cache={}
    def calc(mode, policy, start, stop, label, prices=None, save=False):
        pv=pv2 if mode==2 else pv3; k=(mode, policy.quantile, stop)
        if k not in cache: cache[k]=risk_buffers(data, ll[:stop], pv[:stop], policy.quantile)
        r=run(data, ll, pv, cache[k], policy, start=start, stop=stop,
            initial=6244.256891388887 if start==21 else 9902.811287870369,
            name=label, prices=prices, retain=save)
        if save:
            for obj, suffix in
zip([r[0], r[1], r[2], r[4]], ['daily.csv', 'intervals.csv.gz', 'versions.csv.gz', 'solvers.csv.gz']):
                if len(obj): obj.to_csv(OUT/f'{label}_{suffix}', index=False, float_format='%.17g')

dump_json(OUT/f'{label}_summary.json', r[3]); print(label, round(r[3]['total_cost_yuan'], 2), flush=True)
    return r
    # Fixed design preceding the new evaluation: exact Q3 mechanism, three risks,
    # all update subsets; Q4 fair price comparison with unchanged physical policy.
    design={'q3_quantiles':[0., .5, .7], 'q3_masks':list(range(8)), 'q4_models':['seasonal', 'ridge', 'gru_mean'],
        'q4_2_policy':{'quantile':.7, 'terminal':6000}, 'settlement':'final vs midnight; no fees for
intermediate reversals',
        'scope':'same-year revision; no claim of untouched external test set'}
    dump_json(OUT/'design.json', design); rows=[]
    for q in design['q3_quantiles']:
        for mask in range(8):
            r=calc(3, Policy(quantile=q, mask=mask), 21, 31, 'validation')
            rows.append({'q':q, 'mask':mask, 'cost':r[3]['total_cost_yuan']})
    pd.DataFrame(rows).to_csv(OUT/'q3_validation.csv', index=False)
    best=min(rows, key=lambda x:(x['cost'], x['mask'], x['q'])); pol=Policy(quantile=best['q'], mask=best['mask'])
    pvals=[]
    for mode in [2, 3]:
        for m in design['q4_models']:
            r=calc(mode, Policy() if mode==2 else pol, 21, 31, 'validation', pp[m])
            pvals.append({'mode':mode, 'model':m, 'cost':r[3]['total_cost_yuan']})
    selected={str(mode):min([r for r in pvals if r['mode']==mode], key=lambda x:x['cost'])['model'] for mode
in [2, 3]}

dump_json(OUT/'selection.json', {'q3':best, 'prices':selected}); pd.DataFrame(pvals).to_csv(OUT/'q4_validation.
csv', index=False)
    summaries=[]
    for mask in
range(8): summaries.append(calc(3, Policy(quantile=pol.quantile, mask=mask), 31, 365, f'q3_m{mask}', save=True)[3])
    for mode in [2, 3]:
        for m in design['q4_models']:
            summaries.append(calc(mode, Policy() if mode==2 else pol, 31, 365, f'q4_{mode}_{m}', pp[m], True)[3])
    # Actual execution comparison with clairvoyant prices only (NOT a bound on
    # an unknown global stochastic optimum): future demand/PV remain forecasts.
    oracle=np.full((365, 4, 144), np.nan)
    for v in range(4): oracle[:, v, :144-v*36]=data['actual_price'][:, v*36:]
    for mode in [2, 3]: summaries.append(calc(mode, Policy() if mode==2 else
pol, 31, 365, f'q4_{mode}_oracle', oracle, True)[3])
    dump_json(OUT/'summary.json', summaries)
    # Provable nominal-stage bound: identical 334 forecast feasible sets and
    # initial states for every price predictor, evaluated with true prices.
    states=pd.read_csv(OUT/f'q4_2_{selected["2"]}daily.csv').soc_start_kwh.to_numpy()
    rr=cache[(2, .7, 365)]; regrets=[]
    for i, day in enumerate(range(31, 365)):
        ptrue=data['actual_price'][day]; L=ll[day, 0]+rr[day, 0]; V=pv2[day, 0]
        f, meta=solve(L, V, ptrue, states[i]); lower=float(ptrue@f.plan_kwh)
        for model in ['seasonal', 'ridge', 'gru_mean']:
            plan,_=solve(L, V, pp[model][day, 0], states[i]); cost=float(ptrue@plan.plan_kwh)

```

```

        assert cost>=lower-1e-5

    regrets.append({'date':str(data['dates'][day].date()),'model':model,'nominal_cost':cost,'oracle_lower':lower
    ,'regret':cost-lower})
    pd.DataFrame(regrets).to_csv(OUT/'nominal_price_bound.csv',index=False,float_format='%.17g')
    dump_json(OUT/'run_manifest.json',{'seconds':time.perf_counter()-beg,'input_audit':audit,
    'sources':{f:hashlib.sha256(Path(f).read_bytes().replace(b'\r\n',b'\n')).hexdigest() for f in
    ['src/v3_model.py','src/v3_experiments.py']},
    'bound_scope':'fixed common nominal demand/PV forecasts, risk allowance, initial SOC and terminal
    constraints; no emergency execution claim'})
    print('COMPLETE',round(time.perf_counter()-beg,2),flush=True)

if __name__=='__main__':main()

```

src/v3_verify.py

```
"""Reconstruct final contracts independently and compare all physical traces."""
from pathlib import Path
import argparse, json, hashlib
import numpy as np
import pandas as pd
from src.q12 import audit_trace, dump_json, TOL
from src.q34_data import read_extended

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    out=Path('artifacts/v3'); data,_=read_extended(a.data_root); checks={}
    manifest=json.loads((out/'run_manifest.json').read_text(encoding='utf8'))
    for f,h in manifest['sources'].items(): assert
    hashlib.sha256(Path(f).read_bytes().replace(b'\r\n', b'\n')).hexdigest()==h
    summaries=json.loads((out/'summary.json').read_text(encoding='utf8'))
    for s in summaries:

name=s['strategy']; f=pd.read_csv(out/f'{name}_intervals.csv.gz'); daily=pd.read_csv(out/f'{name}_daily.csv')
    assert len(f)==334*144 and len(daily)==334
    prices=np.tile(data['price'], 334) if name.startswith('q3') else data['actual_price'][:31:].reshape(-1)
    for col, expected in [('load_kwh', data['load'][:31:].reshape(-1)), ('pv_kwh', data['pv'][:31:].reshape(-1)), ('price', prices)]:
        np.testing.assert_allclose(f[col], expected, atol=1e-10, rtol=0)
        physical=audit_trace(f, initial=9902.811287870369)
        vp=out/f'{name}_versions.csv.gz'; vs=pd.read_csv(vp) if vp.exists() else pd.DataFrame()
        groups={d:g for d,g in vs.groupby('date')} if len(vs) else {}
        for date, g in f.groupby('date', sort=False):
            plan=g.original_plan_kwh.to_numpy().copy()
            if date in groups:
                for ver, v in groups[date].groupby('version', sort=True):

slots=v.slot.to_numpy(int); np.testing.assert_array_equal(slots, np.arange(int(ver)*36, 144))
        np.testing.assert_allclose(v.old_kwh, plan[slots], atol=TOL, rtol=0)

np.testing.assert_allclose(v.decision_soc_kwh, g.soc_start_kwh.iloc[int(ver)*36], atol=TOL, rtol=0)
        plan[slots]=v.new_kwh
        np.testing.assert_allclose(plan, g.plan_kwh, atol=TOL, rtol=0)
        # Independent formula uses retained quantity + positive penalties, not
        # the solver's original +/- accounting expression.
        x=f.original_plan_kwh.to_numpy(); y=f.plan_kwh.to_numpy(); u=np.maximum(x-y, 0); v=np.maximum(y-x, 0)

np.testing.assert_allclose(f.decrease_kwh, u, atol=TOL, rtol=0); np.testing.assert_allclose(f.increase_kwh, v, atol=TOL, rtol=0)
        total=prices*(np.minimum(x,y)+.5*u+.5*v+.5*f.emergency_kwh.to_numpy())
        np.testing.assert_allclose(total, f.total_cost_yuan, atol=TOL, rtol=0)
        np.testing.assert_allclose(total.reshape(334, 144).sum(1), daily.total_cost_yuan, atol=1e-6, rtol=0)
        assert abs(total.sum()-s['total_cost_yuan'])<1e-5
        for col in
['original_cost_yuan', 'increase_cost_yuan', 'decrease_cost_yuan', 'emergency_cost_yuan', 'total_cost_yuan']:
            assert abs(f[col].sum()-s[col])<1e-5
        solver=pd.read_csv(out/f'{name}_solvers.csv.gz'); assert (solver.status==0).all()

checks[name]={ 'physical': physical, 'cost': float(total.sum()), 'versions': len(vs), 'solver_calls': len(solver) }
    assert len(checks)==16
    q1=pd.read_csv(out/'q1_intervals.csv'); audit_trace(q1, initial=6000)
    for col, key in
[('load_kwh', 'q1_load'), ('pv_kwh', 'q1_pv'), ('price', 'price')]: np.testing.assert_allclose(q1[col], np.roll(data[key], 1), atol=1e-10)
    b=pd.read_csv(out/'nominal_price_bound.csv'); assert len(b)==1002 and b.regret.min()>-1e-5
    # Paired GRU vs seasonal actual savings with temporal block dependence.
    df0=pd.read_csv(out/'q4_3_seasonal_daily.csv'); df1=pd.read_csv(out/'q4_3_gru_mean_daily.csv')
    diff=(df0.total_cost_yuan-df1.total_cost_yuan).to_numpy(); rng=np.random.default_rng(20260911); cis={}
    for length in [7, 14, 28]:
        starts=rng.integers(0, len(diff), (2000, int(np.ceil(len(diff)/length))))
        ids=((starts[::, None]+np.arange(length))%len(diff)).reshape(2000, -1)[::, :len(diff)]
        cis[str(length)]=np.quantile(diff[ids].sum(1), [.025, .975]).tolist()

dump_json(out/'verification.json', { 'pass': True, 'traces': checks, 'q1_pass': True, 'nominal_bound_cases': len(b),
    'gru_saving_yuan': float(diff.sum()), 'gru_saving_block95': cis, 'daily_series_exact_order': True })
    print('PASS:16 full-year traces, final settlements, Q1 alignment, 1002 paired nominal cases', cis)

if __name__=='__main__': main()
```

src/deliver_q12.py

"""Build the Q1/Q2 manuscript, plotting inputs, figures and Excel payload.

This never refits forecasts or changes dispatch decisions. Inputs are saved results.

"""

```
from __future__ import annotations
import argparse
from dataclasses import replace
import hashlib
import json
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from src.q12 import B, TOL, audit_trace, dump_json, interval_label, optimize_day
```

```
LABELS = {"seasonal_q0": "同期预测 无余量", "ridge_q0": "岭回归 无余量",
          "seasonal_q0.7": "同期预测 70%余量", "ridge_q0.7": "岭回归 70%余量"}
COLORS = {"load": "#3E608D", "pv": "#E8A33D", "pv_fill": "#F0C284", "grid": "#4D779B",
          "soc": "#8074C8", "emergency": "#992224", "forecast": "#7895C1", "actual": "#CD3B42", "base": "#9D9EA3"}
DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
```

```
def md_table(headers, rows):
    def fmt(x):
        return f"{x:,.2f}" if isinstance(x, (float, np.floating)) else str(x)
    return "\n".join([" | ".join(headers) + " |", " | "+" | ".join(["---"] * len(headers)) + " |"] +
                     [" | "+" | ".join(fmt(x) for x in r) + " |" for r in rows])
```

```
def blocks(f):
    return [[f"{h:02d}:{00-{h+4:02d}:00}", float(f.iloc[h*6:(h+4)*6].charge_kwh.sum()),
            float(f.iloc[h*6:(h+4)*6].discharge_kwh.sum())] for h in range(0, 24, 4)]
```

```
def emergency_runs(f):
    positive = f.emergency_kwh.to_numpy() > TOL
    result = []; i = 0
    while i < len(f):
        if not positive[i]:
            i += 1; continue
        j = i + 1
        while j < len(f) and positive[j]: j += 1
        label = interval_label(i).split("-")[0] + "-" + interval_label(j-1).split("-")[1]
        result.append([label, float(f.emergency_kwh.iloc[i:j].sum())]); i = j
    return result
```

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--skill-root", type=Path, default=Path.home() / ".codex/skills/math-modeling")
    args = parser.parse_args(); out = args.out
    sys.path.insert(0, str(args.skill_root / "tools/figure/scripts"))
    from export_figure import export_figure
    from visual_qa import audit_layout
    q1 = pd.read_csv(out / "q1_intervals.csv")
    a = json.loads((out / "q1_summary.json").read_text(encoding="utf-8"))
    summary = json.loads((out / "q2_summary.json").read_text(encoding="utf-8"))
    manifest = json.loads((out / "run_manifest.json").read_text(encoding="utf-8"))
    if manifest["stage"] != "full": raise ValueError("Full-year outputs required")
    selected = summary["selected_strategy"]
    f = pd.read_csv(out / f"{selected}_intervals.csv.gz")
    f["interval_start"] = pd.to_datetime(f.interval_start)
    daily = pd.read_csv(out / "q2_daily_metrics.csv")
    chosen_daily = daily[daily.strategy == selected].set_index("date")
    val = pd.read_csv(out / "policy_validation.csv")
```

```

metrics={r["strategy"]:r for r in summary["strategies"]}
m=metrics[selected];base=metrics["seasonal_q0"];r0=metrics["ridge_q0"]
audit={"q1":audit_trace(q1,initial=6000),"q2":{}},"accounting":{}}
for name,row in metrics.items():
    trace=pd.read_csv(out/f"{name}_intervals.csv.gz")
    audit["q2"][name]=audit_trace(trace,initial=manifest["evaluation_initial_soc"])
    cost=float(np.dot(trace.plan_kwh+5*trace.emergency_kwh,trace.price))
    if abs(cost-row["total_cost_yuan"])>0.02:raise AssertionError("Saved trace accounting differs")
    audit["accounting"][name]={recomputed_cost_yuan:cost,difference_yuan:cost-row["total_cost_yuan"]}
    if len(trace)!=334*144:raise AssertionError("Incomplete trace")
    dump_json(out/"saved_output_audit.json",audit)
    f[f.date.isin(DATES)].to_csv(out/"selected_dates_plot_data.csv",index=False)
    q1_sensitivity=[]
    for eta in (0.9,float(np.sqrt(0.9))):
        _,meta=optimize_day(q1.load_kwh.to_numpy(),q1.pv_kwh.to_numpy(),q1.price.to_numpy(),6000,(6000,"equal"),
            battery=replace(B,eta_c=eta,eta_d=eta))

    q1_sensitivity.append({"eta_charge":eta,"eta_discharge":eta,"round_trip":eta**2,"cost_yuan":meta["cost"]})
    pd.DataFrame(q1_sensitivity).to_csv(out/"q1_efficiency_sensitivity.csv",index=False)
    payload={"q1":{"plan":[[interval_label(i),float(g)] for i,g in enumerate(q1.plan_kwh)],
        "storage":[r+["00:00",6000] if i==0 else ["24:00",6000] if i==1 else [None,None]]
        for i,r in enumerate(blocks(q1))]},
        "q2":{"headers":["日期\\时间"]+[interval_label(i) for i in range(144)]+"全天购电量","全天购电费",
        "plan":[],"storage":[],"emergency":[]}}
    for date,g in f.groupby("date",sort=True):
        s=chosen_daily.loc[date]

    payload["q2"]["plan"].append([date]+g.plan_kwh.astype(float).tolist()+[float(s.plan_kwh),float(s.total_cost_yuan)])
    for i,row in enumerate(blocks(g)):
        payload["q2"]["storage"].append([date if i==0 else None]+row+
            (["00:00",float(s.soc_start_kwh)] if i==0 else ["24:00",float(s.soc_end_kwh)] if i==1 else
            [None,None]))
        runs=emergency_runs(g)
        if not runs:runs=[["无紧急购电",0.0]]
        payload["q2"]["emergency"].extend([[date if i==0 else None]+row for i,row in enumerate(runs)])
    dump_json(out/"xlsx_payload.json",payload)
    pd.DataFrame({"slot":np.arange(144),"input_right_endpoint":[interval_label(i).split("-")[1] for i in
    range(144)],
        "normalized_output_interval":[interval_label(i) for i in
    range(144)]}).to_csv(out/"time_mapping.csv",index=False)
    figdir=Path("figures/q12");figdir.mkdir(parents=True,exist_ok=True)
    plt.rcParams.update({"font.sans-serif":["Microsoft YaHei","SimHei","DejaVu
    Sans"],"axes.unicode_minus":False,
    "font.size":10,"axes.spines.top":False,"axes.spines.right":False,"svg.fonttype":"none"})
    figure_audit={}
    def save(fig,name):
        issues=audit_layout(fig)
        if any(level in ("FAIL","WARN") for level,_ in issues):raise RuntimeError(f"Figure layout: {issues}")
        figure_audit[name]=issues

    export_figure(fig,str(figdir/name),formats=["png","svg"],dpi=300,size_inches=tuple(fig.get_size_inches()),
        grayscale_preview=False,tight=False)
    svg=figdir/(name+".svg")
    svg.write_text("\n".join(line.rstrip() for line in svg.read_text(encoding="utf-
    8")).splitlines()+"\n",
        encoding="utf-8",newline="\n")
    plt.close(fig)
    x=(np.arange(144)+0.5)/6

    fig,axs=plt.subplots(2,1,figsize=(7.2,5.8),sharex=True,layout="constrained",gridspec_kw={"height_ratios":[2,
    1]})
    for key,label,color in [("load_kwh","负载",COLORS["load"]),("pv_kwh","光伏",COLORS["pv"]),("plan_kwh","计
    划购电",COLORS["grid"])]):
        axs[0].plot(x,q1[key]*6,label=label,color=color,lw=1.5,ls="--" if key=="plan_kwh" else "-")
        axs[0].set_ylim(0,1.25*q1[["load_kwh","pv_kwh","plan_kwh"]].to_numpy().max()*6)
        axs[0].set_ylabel("功率 / kW");axs[0].legend(ncol=3,loc="upper left");axs[0].set_title("(a) 单日供需与购电",loc="left",fontsize=11)
        axs[1].plot(np.arange(145)/6,np.r_[q1.soc_start_kwh.iloc[0],q1.soc_end_kwh],color=COLORS["soc"])
        axs[1].axhline(1200,color=COLORS["base"],ls=":");axs[1].axhline(10800,color=COLORS["base"],ls=":")
        axs[1].set_ylabel("储电量 / kWh");axs[1].set_xlabel("时刻 / h");axs[1].set_xticks(np.arange(0,25,4));axs[1].set_xlim(0,24)
        axs[1].set_title("(b) 储能状态",loc="left",fontsize=11);save(fig,"q1_dispatch")
    fig,axs=plt.subplots(2,1,figsize=(7.2,5.4),sharex=True,layout="constrained")
    g=f[f.date==DATES[0]]
    for ax,key,label in zip(axs,("load","pv"),("负载","光伏")):

```

```

ax.plot(x,g[key+"_kwh"]*6,color=COLORS["actual"],label="实际值",lw=1.4)
ax.plot(x,g["forecast_"+key+"_kwh"]*6,color=COLORS["forecast"],label="日前预测",ls="--",lw=1.5)
ax.set_ylabel(label+" /
kw");ax.set_ylim(0,1.25*g[[key+"_kwh"],"forecast_"+key+"_kwh"].to_numpy().max()*6)
axs[0].legend(ncol=2,loc="upper left");axs[0].set_title("2025-03-20 岭回归日前预测",loc="left",fontsize=11)
axs[1].set_xlabel("时刻 / h");axs[1].set_xticks(np.arange(0,25,4));axs[1].set_xlim(0,24);save(fig,"q2_forecast")
names=list(metrics);idx=np.arange(len(names))
fig,ax=plt.subplots(figsize=(7.2,4.5),layout="constrained")
normal=np.array([metrics[n]["normal_cost_yuan"] for n in names])/1e4
emerg=np.array([metrics[n]["emergency_cost_yuan"] for n in names])/1e4
ax.bar(idx,normal,color=COLORS["grid"],label="计划购电费",width=.55)
ax.bar(idx,emerg,bottom=normal,color=COLORS["emergency"],label="紧急购电费",width=.55)
ax.set_xticks(idx,[LABELS[n].replace(" ","\n") for n in names]);ax.set_ylabel("2-12月费用 / 万元")
ax.set_ylim(0,max(normal+emerg)*1.23);ax.legend(ncol=2,loc="upper right")
for i,total in enumerate(normal+emerg):ax.text(i,total+20,f"{total:,.1f}",ha="center",fontsize=10)
save(fig,"q2_cost_comparison")
fig,axs=plt.subplots(1,2,figsize=(7.2,3.5),layout="constrained")
for method,color in [{"seasonal",COLORS["base"]},("ridge",COLORS["load"])]):
    v=val[val.forecast==method]
    label="同期预测" if method=="seasonal" else "岭回归"
    axs[0].plot(range(4),v.cost_yuan/1e4,color=color,marker="o",ms=4,label=label)
    axs[1].plot(range(4),v.emergency_cost_yuan/1e4,color=color,marker="s",ms=4,label=label)
for ax in axs:
    ax.set_xticks(range(4),["无余量","70%","80%","90%"]);ax.set_xlabel("历史误差分位数");ax.set_ylim(bottom=0)
axs[0].set_ylabel("验证期总费用 / 万元");axs[1].set_ylabel("验证期紧急费用 / 万元")
axs[0].legend(loc="lower left",fontsize=9);save(fig,"q2_validation")
dump_json(out/"figure_layout_audit.json",figure_audit)
q1table=md_table(["时段","计划购电量 / kWh"],[[interval_label(h*6),float(q1.plan_kwh.iloc[h*6])] for h in (10,12,14,16,18,20)])
q1storage=md_table(["时段","充电量 / kWh","放电量 / kWh"],blocks(q1))
comparison=md_table(["策略","计划费 / 万元","紧急费 / 万元","总费用 / 万元","紧急购电 / kWh"],
[[LABELS[n],metrics[n]["normal_cost_yuan"]/1e4,metrics[n]["emergency_cost_yuan"]/1e4,metrics[n]["total_cost_yuan"]/1e4,metrics[n]["emergency_kwh"] for n in names])
forecast_table=md_table(["预测器","负载 MAE / kW","负载 RMSE / kW","光伏 MAE / kW","光伏 RMSE / kW"],
[["同期预测",base["load_mae_kw"],base["load_rmse_kw"],base["pv_mae_kw"],base["pv_rmse_kw"]],
["岭回归",m["load_mae_kw"],m["load_rmse_kw"],m["pv_mae_kw"],m["pv_rmse_kw"]]])
date_summary=md_table(["日期","全天计划购电 / kWh","全天总购电费 / 元","紧急购电 / kWh","日初 SOC / kWh","日末 SOC / kWh"],
[[d,*[float(chosen_daily.loc[d,k]) for k in ["plan_kwh","total_cost_yuan","emergency_kwh","soc_start_kwh","soc_end_kwh"]] for d in DATES])
# Build text from computed evidence. Mathematics stays editable in Markdown.
text=rfr'# 考虑预测误差的微网日前购电与储能调度

```

第一问与第二问初稿

摘要

针对小区光伏发电、负载需求与分时电价共同影响的微网购电问题，建立统一的能量平衡与储能状态模型。在给定单日曲线时，以计划购电费最小为目标，采用混合整数线性规划约束充放电互斥，并要求日初日末储能电量相等。对于未来供需未知的情形，比较同期预测和岭回归，使用历史净负载误差的分位数构造非负余量，再通过实际运行仿真评估计划费用与五倍电价的紧急购电费用。

在充电、放电效率分别为 90%，输入功率视为十分钟区间平均值的假设下，第一问全天计划购电 $\{a['plan_kwh'] : .2f\}$ kWh，购电费 $\{a['cost'] : .2f\}$ 元，相对无储能基线降低 $\{a['saving_percent'] : .2f\}\%$ 。第二问利用一月份验证数据选择岭回归与 70% 分位数余量，随后在 2-12 月 334 天进行顺序回溯。该策略总费用为 $\{m['total_cost_yuan']/1e4 : .2f\}$ 万元，相对同期预测无余量基线下降 $\{100*(1-m['total_cost_yuan']/base['total_cost_yuan']) : .2f\}\%$ ；相对岭回归无余量策略下降 $\{100*(1-m['total_cost_yuan']/r0['total_cost_yuan']) : .2f\}\%$ 。结果表明，预测改进与适量风险余量均能降低费用，但更高余量虽可进一步减少紧急购电，也可能增加总成本。本稿仅回答前两问，其结论限于所列信息、时间、效率与运行假设。

关键词：微网调度；混合整数线性规划；岭回归；风险余量；时间顺序回溯

1 问题分析

题面及附件提供了微网的物理参数、供需曲线与结果模板 [1]。第一问的电价、负载和光伏预测曲线已给定，需决定各时段的购电与储能安排，属于确定性优化问题，不需要拟合预测模型。储能的经济作用来自跨时段转移电量，必须同时考虑功率上限、容量边界和能量损失。

第二问要求每天午夜制定整天计划。实际用电和发电变化后，普通购电仍按计划量付费，剩余供电缺口通过五倍电价紧急补购。因此，

少买与多买均存在代价。模型分为日前预测、计划优化和实时执行三个部分；历史真实值用于过去信息与事后评价，不能用于提前决定当天购电量。

2 数据处理与模型假设

附件 1 含 144 个十分钟时刻，附件 2 的负载与光伏各含 $365 \times 144 = 52,560$ 个观测。实际读取未发现缺失值、负数或日期重复，保留全部样本，不对波峰进行无依据的删除或平滑。附件 1 部分时间单元格为字符串、部分为时间类型，读取后统一转换为分钟数，检查为 10、20、...、1440 的完整序列。

采用下列假设，并将它们与题面直接给定的参数区分：

1. 每个功率值表示以其时间标签为右端点的十分钟区间平均功率。电量为功率乘 $1/6$ 小时。模板的区间标签存在疑似错位，本稿按内部区间起止时间汇总，导出副本同步采用规范标签，原始附件保持不变。
2. 充电和放电效率各取 0.9，往返效率为 0.81；功率上限定义在微网母线侧。另检验总往返效率 0.9 的解释。
3. 不设置题面未给出的售电收入，允许无法利用的剩余电量被弃用。计划富余电量仍付费。
4. 实时控制采用每十分钟内功率恒定且可即时平衡的近似。当前区间缺口可以被电池和紧急购电补足；这不是提前知道整天真实曲线。
5. 第二问实际 SOC 跨日连续，日前预测轨迹的日末 SOC 下限取 6000 kWh，作为避免耗空电池的策略设置，不强制实际日末回到此值。评价期初值由统一的一月份因果热身策略得到，不从二月起每天重置。

记区间负载与光伏电量为 L_t, V_t ，计划购电为 g_t ，紧急购电为 e_t ，母线侧充电与放电为 c_t, d_t ，未利用电量为 w_t ，区间起点储电量为 S_t 。上述电量单位均为 kWh，电价 p_t 单位为元/kWh。

3 第一问的确定性优化模型

3.1 目标与约束

最小化全天购电费用：

$$\min C_1 = \sum_{t=1}^{144} p_t g_t.$$

第一问不设置紧急购电，能量平衡与储能状态满足：

$$g_t + V_t + d_t = L_t + c_t + w_t, \quad w_t \geq 0,$$

$$S_{t+1} = S_t + 0.9c_t - \frac{d_t}{0.9}, \quad 1200 \leq S_t \leq 10800.$$

令 $z_t \in \{0, 1\}$ 表示充电状态，母线侧每区间最大充放电量为 $M = 5000/6$ kWh，规定

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad g_t \geq 0.$$

取 $S_1 = S_{145} = 6000$ kWh。该等式避免利用初始存量净放电制造虚假的节省。模型共有 865 个变量，其中 144 个为二元变量；利用 SciPy 的 MILP 接口调用 HiGHS 求解 [2]。

3.2 求解结果

全天计划购电量为 $\{a['plan_kwh']\}$ kWh，购电费为 $\{a['cost']\}$ 元。无储能基线按正净负载直接购电，费用为 $\{a['no_storage_cost']\}$ 元，采用储能后降低 $\{a['saving_percent']\}$ %。表 1 给出题目指定区间，表 2 给出四小时充放电汇总。图 1 展示供需、购电与储能轨迹。

表 1 第一问指定区间计划购电量

{q1table}

表 2 第一问储能运行汇总

{q1storage}

00:00 与 24:00 的储电量均为 6000.00 kWh。充电量记录进入电池前的母线电量，放电量记录电池返回母线的电量，二者不应直接按无损储能作差。

![图 1 第一问单日调度与储电量](../figures/q12/q1_dispatch.png)

图 1 第一问单日供需、购电与 SOC 轨迹

3.3 结果检验与效率敏感性

整数模型的目标值与 LP 松弛下界之差为 $\{a['lp_gap_yuan']:.8f\}$ 元，求解器报告相对间隙为 $\{a['mip_gap']:.2g\}$ 。因此，在当前输入与约束口径下，第一问已获得数值精度内的全局最优解。最大能量平衡残差约 $\{a['audit']['max_balance_residual_kwh']:.2e\}$ kWh，未出现同时充放电。

若“效率 90%”解释为总往返效率，将两侧效率均设为 $\sqrt{0.9}$ ，第一问费用变为 $\{q1_sensitivity[1]['cost_yuan']:.2f\}$ 元。由此可见，效率定义会影响数值结论，后续整篇论文应保持同一口径，不混用两组结果。

4 第二问的预测与风险余量模型

4.1 时间顺序与预测器

以 1 月 22–31 日为验证区间，模型只用各预测时点之前已结束的日期训练。岭回归从积累足够历史后开始，每七天重新拟合一次，最多使用最近 60 天。样本不足时采用简单历史曲线。验证期选择完成后，将规则固定并应用到 2–12 月，后续已观测数据可以进入下一次更新，但不回写历史预测。

同期基线使用昨日和上周同日相同时段的均值。岭回归使用昨日、前日、上周同日的同槽电量，最近三日与七日同槽均值、七日标准差、过去日均值，三阶日内正余弦项以及星期指示变量，共 21 个特征。所有标准化参数均由当前训练段计算，预测当日的真实负载与光伏不进入特征。

对负载和光伏分别拟合带截距的岭回归 [3]：

$$\min_{\{\beta, b\}} \sum_i (y_i - b - x_i^T \beta)^2 + \lambda \|\beta\|_2^2.$$

截距不参与惩罚。验证比较 $\lambda \in \{1, 10, 100\}$ ，两种变量均选择 $\lambda = 1$ 。预测值截断为非负；如果某时槽过去七天的光伏均为零，则其预测置零。该规则只利用历史，但对日出日落变化的适应存在滞后。

4.2 风险余量

令净负载预测误差为

$$\varepsilon_{d,t} = (L_{d,t} - V_{d,t}) - (\hat{L}_{d,t} - \hat{V}_{d,t}).$$

对决策日之前最近 28 天、目标时槽前后三个十分钟槽内的历史误差取经验分位数，并截断为非负，得到余量 $r_{d,t}(q)$ 。不足 28 天时只使用已有历史，时槽池化不跨越日界。初稿比较无余量以及 $q = 0.7, 0.8, 0.9$ 。

将 $\hat{L} + r(q)$ 与 $\hat{V}$ 代入第一问的物理模型，以计划购电费最小求得日前计划，同时将预测轨迹日末储电量约束为至少 6000 kWh。这里的分位数余量是一种可解释的风险控制近似，**不是精确求解全年随机最优控制，也不等于以概率 q 保证全日无缺口**。

4.3 实时执行与费用

当天计划确定后不再倒改。每一十分钟区间，令计划购电与实际光伏减去实际负载的差额为 u_t 。当 $u_t \geq 0$ 时，在功率和容量允许范围内充电，剩余部分记为未利用电量；当 $u_t < 0$ 时，先按功率和 SOC 限制尽可能放电，剩余缺口记为紧急购电。所有区间均满足供电平衡，实际 SOC 连续传递至次日。

实际评价费用为

$$C_2 = \sum_t p_{tg,t} \{plan\} + \sum_t p_{te,t}.$$

普通购电按计划量收费，未利用的计划电量不退款。余量参数按验证期的该项总费用选择，而不是只按预测误差或紧急购电量选择。

5 第二问结果与分析

5.1 验证期的费用权衡

图 2 比较不同余量的验证费用。岭回归配合 70% 分位数余量的十天总费用为 {summary['selection']['cost_yuan']:, .2f} 元，紧急费用为 {summary['selection']['emergency_cost_yuan']:, .2f} 元，在候选方案中最低。岭回归配合 90% 余量的验证期紧急费用为零，但总费用为 {float(val[(val.forecast=='ridge') & (val['quantile']==0.9)].cost_yuan.iloc[0]):, .2f} 元，更高的计划支出抵消了避免紧急购电的收益。

![图 2 验证期余量与费用](../figures/q12/q2_validation.png)

图 2 一月验证期风险余量与费用的关系

四种策略在评价期采用相同初始 SOC: {manifest['evaluation_initial_soc']:, .2f} kWh，来源于从 1 月 1 日 6000 kWh 出发的共同热身运行。不同策略在后续产生各自的 SOC 轨迹，没有按天重置。验证期与全年比较均报告期末存量，避免忽略存量差异。

5.2 样本外预测与调度效果

表 3 为 334 天共 48,096 个时段上的预测误差。相对同期基线，岭回归对负载的改善较明显，光伏改善较小。图 3 展示题目指定的 3 月 20 日预测结果，单日图用于说明误差形态，整体评价以全期指标为准。

表 3 第二问样本外预测误差

{forecast_table}

![图 3 指定日期的负载与光伏预测](../figures/q12/q2_forecast.png)

图 3 2025 年 3 月 20 日的日前预测与实际观测

表 4 与图 4 给出各策略的费用。岭回归无余量相对同期无余量已降低成本；在相同岭回归预测上增加 70% 余量，普通购电费增加，但紧急费用下降更多，最终总费用进一步降低 {100*(1-m['total_cost_yuan']/r0['total_cost_yuan']):, .2f}%。这说明本题需要评价预测与调度的组合，而非只评价预测器。

表 4 第二问 2-12 月策略比较

{comparison}

![图 4 全期费用分解](../figures/q12/q2_cost_comparison.png)

图 4 不同预测与余量策略的全期费用分解

选定策略累计紧急购电 {m['emergency_kwh']:, .2f} kWh，涉及 {m['emergency_intervals']} 个十分钟区间。未利用电量为 {m['spill_kwh']:, .2f} kWh，其中可能同时包含光伏富余和未吸收计划电量，当前账本不将其全部解释为弃光。该结果没有实现全年零紧急购电，也没有证明其为所有可行策略中的全局最优。

岭回归无余量与带余量策略在 12 月 31 日结束时均为 {m['final_soc_kwh']:, .2f} kWh，因此二者的主要费用差异不来自不同的期末电池存量。同期两组的期末 SOC 分别为 {base['final_soc_kwh']:, .2f} 与 {metrics['seasonal_q0.7']['final_soc_kwh']:, .2f} kWh；跨预测器的费用比较仍需注意这种存量差异。

5.3 指定日期结果

表 5 为题目指定四日的汇总。“全天购电量”在本稿及导出表中指普通计划购电量；紧急购电另列。“全天购电费”包含普通与紧急两部分。后续各表给出指定十分钟槽与四小时充放电量。

表 5 第二问指定日期汇总

{date_summary}

...

table_number=6

```

for date in DATES:
    g=f[f.date==date]
    purchases=[[interval_label(h*6),float(g.plan_kwh.iloc[h*6])] for h in (10,12,14,16,18,20)]
    text+=f"### {date}\n\n 表 {table_number} 给出指定时段购电量, 表 {table_number+1} 为储能汇总, 表
{table_number+2} 为连续紧急购电区间。 \n\n"
    text+=f"表 {table_number} {date} 指定区间计划购电\n\n"+md_table(["时段","计划购电 /
kWh"],purchases)+"\n\n"
    text+=f"表 {table_number+1} {date} 四小时充放电\n\n"+md_table(["时段","充电 / kWh","放电 /
kWh"],blocks(g))+"\n\n"
    text+=f"表 {table_number+2} {date} 紧急购电\n\n"+md_table(["时段","紧急购电 / kWh"],emergency_runs(g)
or [{"无",0.0}])+"\n\n"
    table_number+=3
    text+=rf'""## 6 模型评价与后续改进

```

第一问的混合整数模型可直接对应题面物理约束, 并通过 LP 下界和数值残差检验最优性与可行性。第二问将预测、风险余量和实际执行分开, 保持历史信息边界, 并以真实账本评价经济性, 具有可解释和易复现的优点。

当前局限包括: 一月份验证区间较短, 无法充分代表全部季节; 经验分位数没有给出联合概率保证; 贪心实时平衡不一定是最优电池控制; 名义日末 6000 kWh 的目标尚未系统调优; 效率、时间标签及实时观测近似会影响结果。较优的全年表现来自本次实际回测, 不应推广为任意未来数据下的保证。

在本机的一次运行中, 包括预测候选验证、共同热身、策略筛选和四组全年回测的计算约为 {manifest['total_seconds']:.1f} 秒。每组全年共求解 334 个小规模日模型; 该时间不包括代码开发、排错、写作与图表导出, 也不代表第三、四问或更大场景模型的运行耗时。

后续可在不改变费用和物理口径的前提下, 比较更充分的季节验证、终端储能目标、分位数预测与场景优化, 并将第三问的日内预报更新接入同一调度框架。本稿不包含第三、四问结果。

参考文献

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题 微网与外部电网电力调控策略. 用户提供题面及附件 1、2、5.

[2] SciPy Developers. scipy.optimize.milp. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html> . 本文实际运行版本 1.17.1; 在线文档用于接口与方法说明。

[3] scikit-learn Developers. Ridge. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html . 本文按带截距的 L2 正则化目标以 NumPy 实现, 未调用 scikit-learn 软件包。

```

'''
    # The abstract already identifies scope; keep project control notes outside manuscript.
    reportdir=Path("reports");reportdir.mkdir(exist_ok=True)
    (reportdir/"Q1_Q2 初稿.md").write_text(text,encoding="utf-8")
    dump_json(out/"delivery_manifest.json",{ "manuscript": "reports/Q1_Q2 初稿.md", "selected_strategy": selected,
    "figure_count": 4, "table_count": table_number-
1, "builder_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
    "skill_repo": "XiaoMaColtAI/math-modeling-
skill", "skill_commit": "3527fad922660397834a6167fc2b4c29ee64ba17"})
    print("Manuscript and plotting evidence built.",flush=True)

```

```

if __name__=="__main__":main()

```

src/v3_payload.py

```
"""Exact result-cell payloads; preserve populated V2 workbook layouts."""
from pathlib import Path
import json
import pandas as pd
from src.q12 import dump_json
from src.deliver_q12 import blocks, emergency_runs

def main():
    out=Path('artifacts/v3');sel=json.loads((out/'selection.json').read_text());payload={}
    chosen={'3':f'q3_m{sel["q3"]["mask"]}', '4-2':f'q4_2_{sel["prices"] ["2"]}', '4-3':f'q4_3_{sel["prices"] ["3"]}' }
    for key,name in chosen.items():

f=pd.read_csv(out/f'{name}_intervals.csv.gz');plans=[];adjusted=[];storage=[];emergency=[];ledger=[];version_rows=[]
    for date,g in f.groupby('date',sort=False):

plans.append([date,*g.original_plan_kwh.to_list(),float(g.original_plan_kwh.sum()),float(g.total_cost_yuan.sum() if key=='4-2' else g.original_cost_yuan.sum())])

adjusted.append([date,*g.plan_kwh.to_list(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
    for i,b in enumerate(blocks(g)):
        storage.append([date if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else None,float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else None])
    for i,b in enumerate(emergency_runs(g) or [['无',0]]):emergency.append([date if i==0 else None,*b])
    ledger.append([date,*[float(g[c].sum()) for c in ['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
    vp=out/f'{name}_versions.csv.gz'
    if vp.exists():
        v=pd.read_csv(vp)
        for (date,ver),g in v.groupby(['date','version'],sort=True):
            values=[None]*144
            for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
            version_rows.append([date,f'{int(ver)*6:02d}:00',*values])

payload[key]={ 'plan':plans, 'adjusted':adjusted, 'storage':storage, 'emergency':emergency, 'ledger':ledger, 'versions':version_rows}
    q1=pd.read_csv(out/'q1_intervals.csv');b=blocks(q1)
    # Official Q1 template is ordered 00:10,...,24:00. Its last slot is the
    # first slot of the repeated day. Keep every original template label.
    payload['1']={ 'plan':[[float(q1.plan_kwh.iloc[i%144])] for i in range(1,145)],
        'storage':[[*b[i][1:],float(q1.soc_start_kwh.iloc[0]) if i==0 else float(q1.soc_end_kwh.iloc[-1]) if i==1 else None] for i in range(6)]}

dump_json(out/'xlsx_payload.json',payload);dump_json(out/'delivery_selection.json',{'1':'q1_intervals.csv','2':'../v2/result2.xlsx',**chosen})

if __name__=='__main__':main()
```

scripts/export_v3_xlsx.mjs

```
import fs from 'node:fs/promises';
import path from 'node:path';
import {FileBlob, SpreadsheetFile} from '@oai/artifact-tool';
const root=process.argv[2];if(!root)throw Error('Original data root required');
const out='artifacts/v3',p=JSON.parse(await fs.readFile(`${out}/xlsx_payload.json`,`utf8`));
const dates=rows=>rows.map(r=>[r[0]?new Date(`${r[0]}T00:00:00Z`):null,...r.slice(1)]);
await fs.mkdir(`${out}/v3/xlsx',{recursive:true});const audit=[];
for(const key of ['1','3','4-2','4-3']){
  const source=key==='1'?path.join(root,'附件/附件 5/result1.xlsx'):`artifacts/v2/result${key}.xlsx`;
  const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(source));
  if(key==='1'){
    wb.worksheets.getItem('计划购电量').getRange('B2:B145').values=p[key].plan;
    const s=wb.worksheets.getItem('充放电量');s.getRange('B2:C7').values=p[key].storage.map(r=>r.slice(0,2));
    s.getRange('E2:E3').values=p[key].storage.slice(0,2).map(r=>r[2]);
  }else{
    for(const [sheet,field,last] of [['计划购电量','plan','EQ'],['调整购电量','adjusted','EQ'],['充放电量','storage','F'],['紧急购电量','emergency','C'],['费用分解','ledger','F'],['计划版本','versions','EP']]){
      if((sheet==='调整购电量' || sheet==='计划版本')&&key==='4-2')continue;
      const s=wb.worksheets.getItem(sheet),rows=p[key][field];
      // Only data cells are overwritten. Header text, interval labels in the
      // existing V2 filled templates, widths and formatting remain untouched.
      if(field==='storage' || field==='emergency' || field==='versions'){
        const previous=(await s.getUsedRange().values).length;
        if(previous>1)s.getRange(`A2:${last}${previous}`).clear({applyTo:'contents'});
      }
      s.getRange(`A2:${last}${rows.length+1}`).values=dates(rows);
    }
    const note=wb.worksheets.getItem('口径说明');
    if(key!=='4-2')note.getRange('B4:B5').values=[['午夜原计划与最终有效量结算：保留量 1 倍、取消量 0.5 倍、新增量 1.5 倍、紧急 5 倍；中途版本不重复收费'],['最终有效普通计划：每次已执行区间冻结，版本只用于追溯']];
  }
  wb.recalculate();await (await SpreadsheetFile.exportXlsx(wb)).save(`${out}/result${key}.xlsx`);
  const img=await wb.render({sheetName:'计划购电量',range:key==='1'?`A59:B65`:`A1:F5`,scale:1.5,format:'png'});
  await fs.writeFile(`${out}/v3/xlsx/${key}.png`,new Uint8Array(await img.arrayBuffer()));
  audit.push({key,source,template_headers_written:false});console.log(`Exported ${key}`);
}
await fs.copyFile('artifacts/v2/result2.xlsx',`${out}/result2.xlsx`);
await fs.writeFile(`${out}/xlsx_build.json`,JSON.stringify(audit,null,2));
```